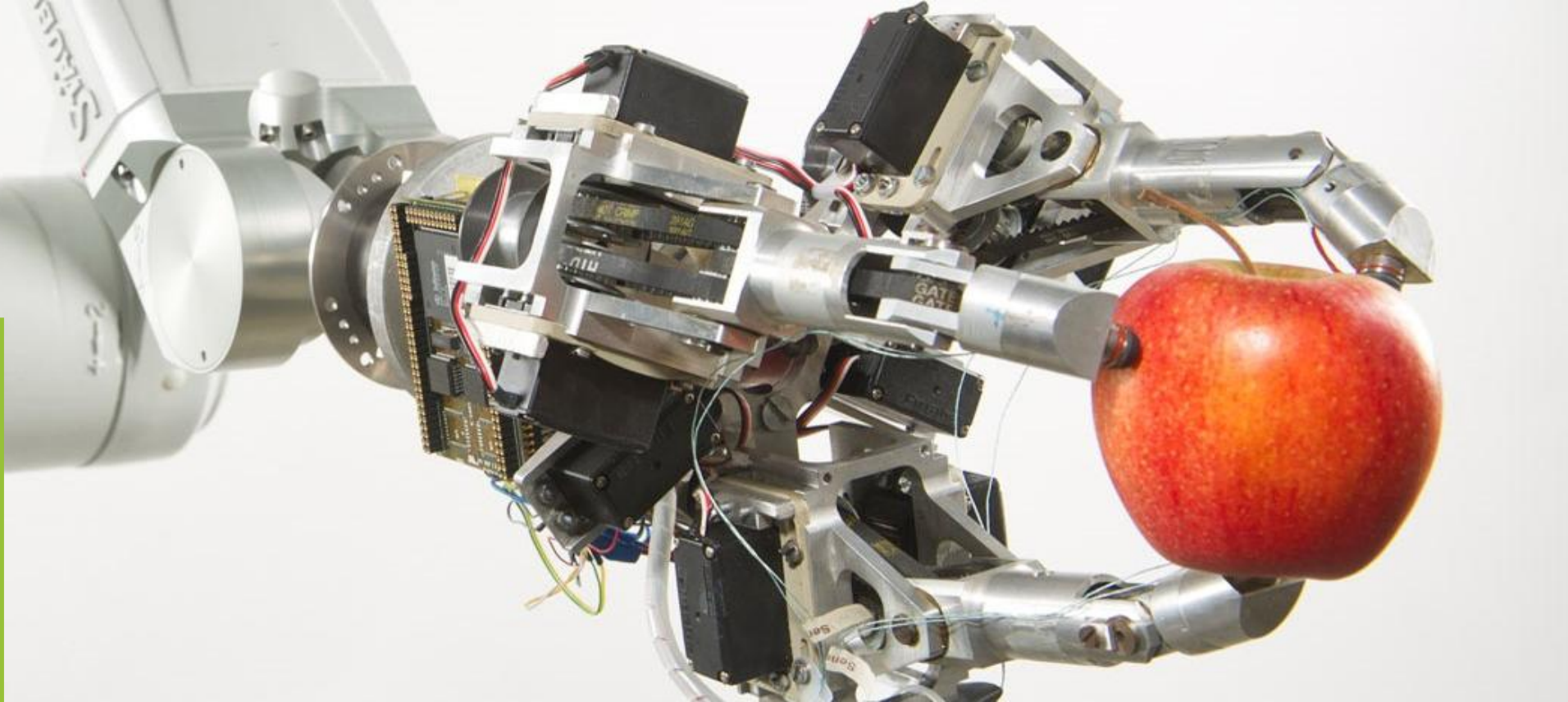




Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

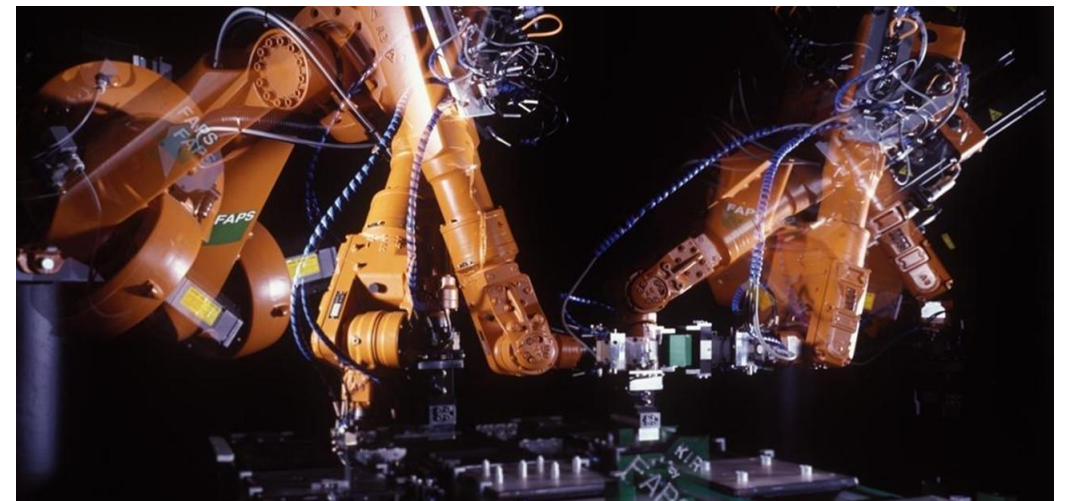
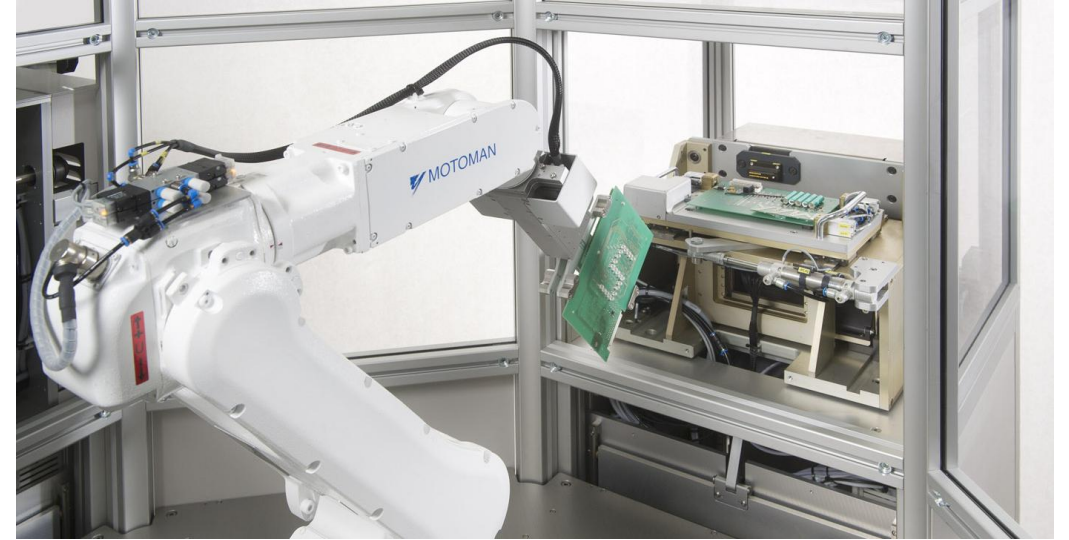
Grundlagen der Robotik (GdR)

VE 10 - Maschinelles Lernen

Im Fach Grundlagen der Robotik werden die fachspezifischen Grundlagen zur Konzeption, Implementierung und Realisierung von Robotersystemen vermittelt.

VE	Thema
----	-------

- | | |
|----|--|
| 1 | Einführung und Grundlagen der Industrierobotik & Neuartige Konzepte und Kinematiken in der Robotik |
| 2 | Steuerung - Sensorik – Aktorik |
| 3 | Roboterkinematik |
| 4 | Roboterdynamik |
| 5 | Bahnplanung und Regelung |
| 6 | Grundlagen des Robot Operating System (ROS) |
| 7 | ROS - Erweiterte Konzepte und Tools |
| 8 | Simulation und Planung |
| 9 | Rechnersehen |
| 10 | Maschinelles Lernen |
| 11 | Seilrobotik |
| 12 | Service-, Pflege- und Medizinrobotik |

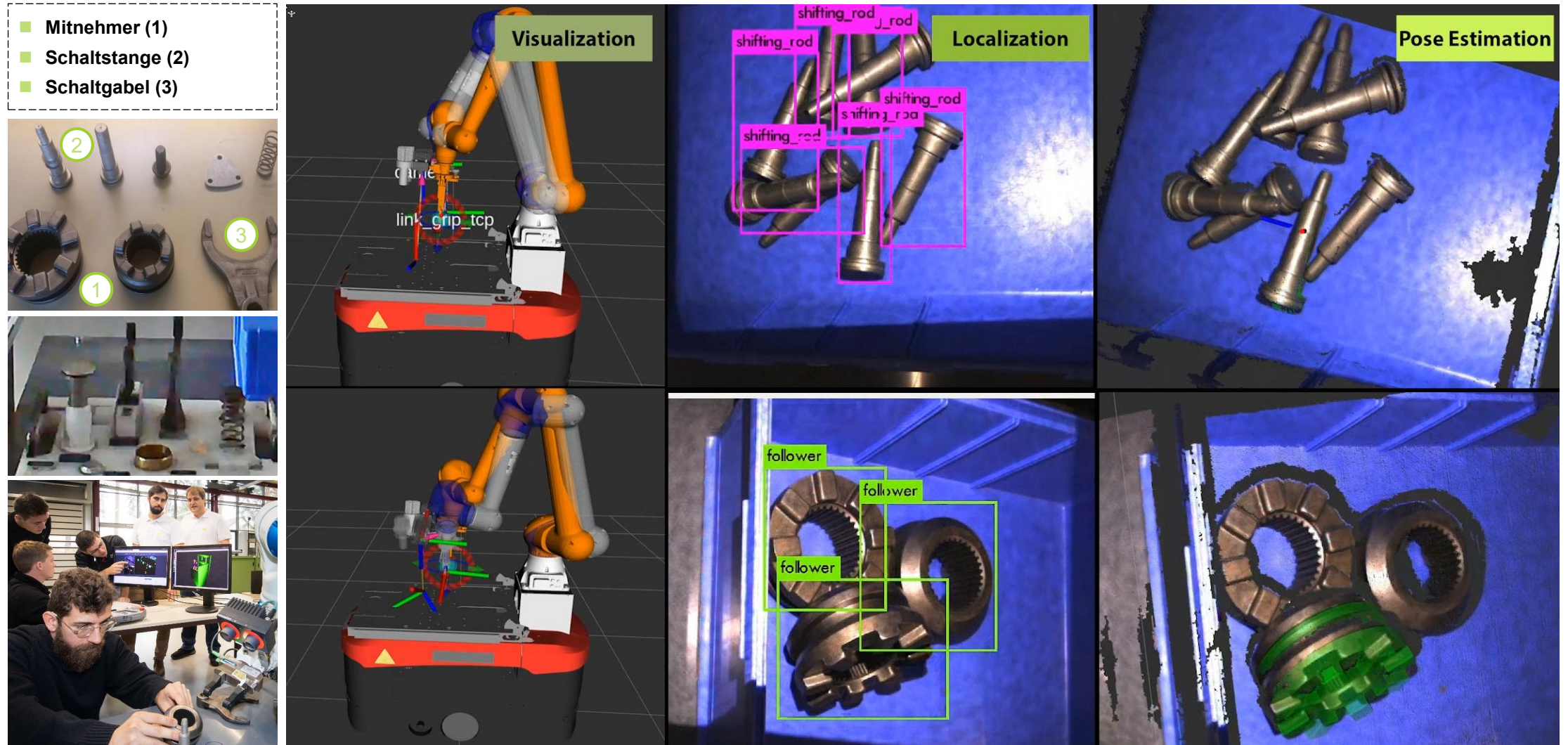


Methoden des maschinellen Lernen ermöglichen
die robuste Detektion vielfältiger, komplexer Objekte.



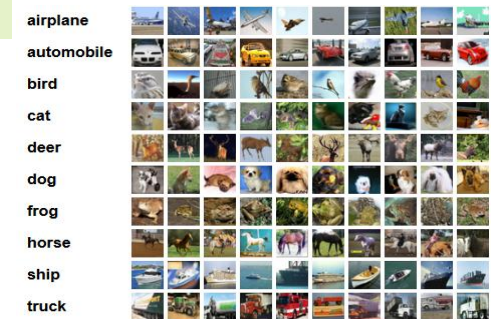
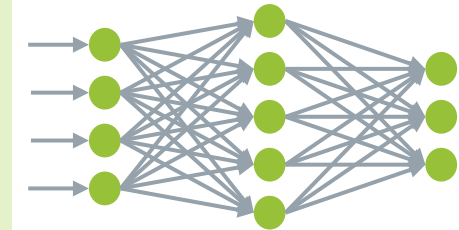
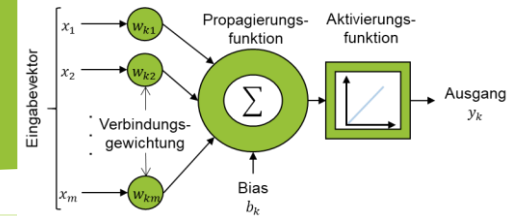
Videoquelle: <http://pjreddie.com/yolo>

Eine kombinierte Pipeline zur 6DoF-Poseschätzung ermöglicht einen robusten und zeiteffizienten Griff in die Kiste für texturlose, metallische Industriekomponenten.



Die Vorlesungseinheit „Maschinelles Lernen“ soll einen Einblick in das Maschinelle Lernen sowie einen Überblick Künstliche Neuronale Netze vermitteln.

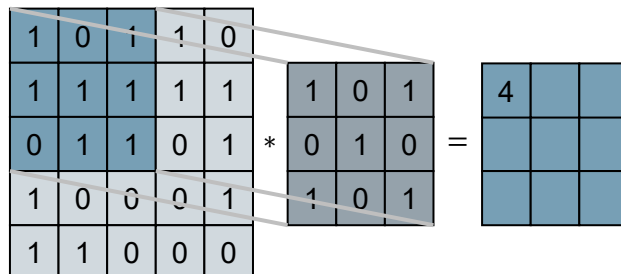
Maschinelles Lernen beschreibt das **automatisierte Lernen und Wiedererkennen von Mustern in Daten** und ist ein Teilgebiet der **künstlichen Intelligenz**.



Bildquelle: <https://www.cs.toronto.edu/~kriz/cifar.html>

Nach dem Besuch dieser Vorlesungseinheit sollten Sie in der Lage sein:

- **Grundlegende maschinelle Lernprinzipien** unterscheiden können
- **Ziele und etablierte Methoden** des maschinellen Lernen kennen
- Relevante Eigenschaften und Parameter von **künstlichen neuronalen Netzen** kennen
- Grundprinzip des **Lernverfahrens Backpropagation und Gradientenabstiegsverfahren** beschreiben können
- Grundprinzipien von **faltenden neuronalen Netzen** beschreiben können



Gliederung VE10

„Maschinelles Lernen“

Motivation und Überblick

Grundlagen Maschinelles Lernen

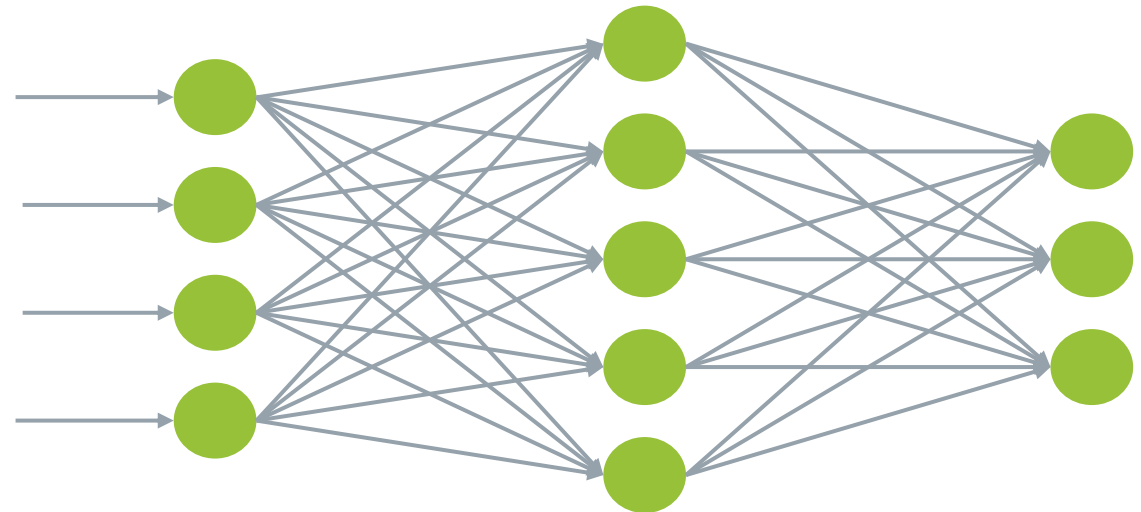
- Grundprinzipien und Ziele des maschinellen Lernens
- Grundlegende etablierte Algorithmen des maschinellen Lernens

Künstliche Neuronale Netze

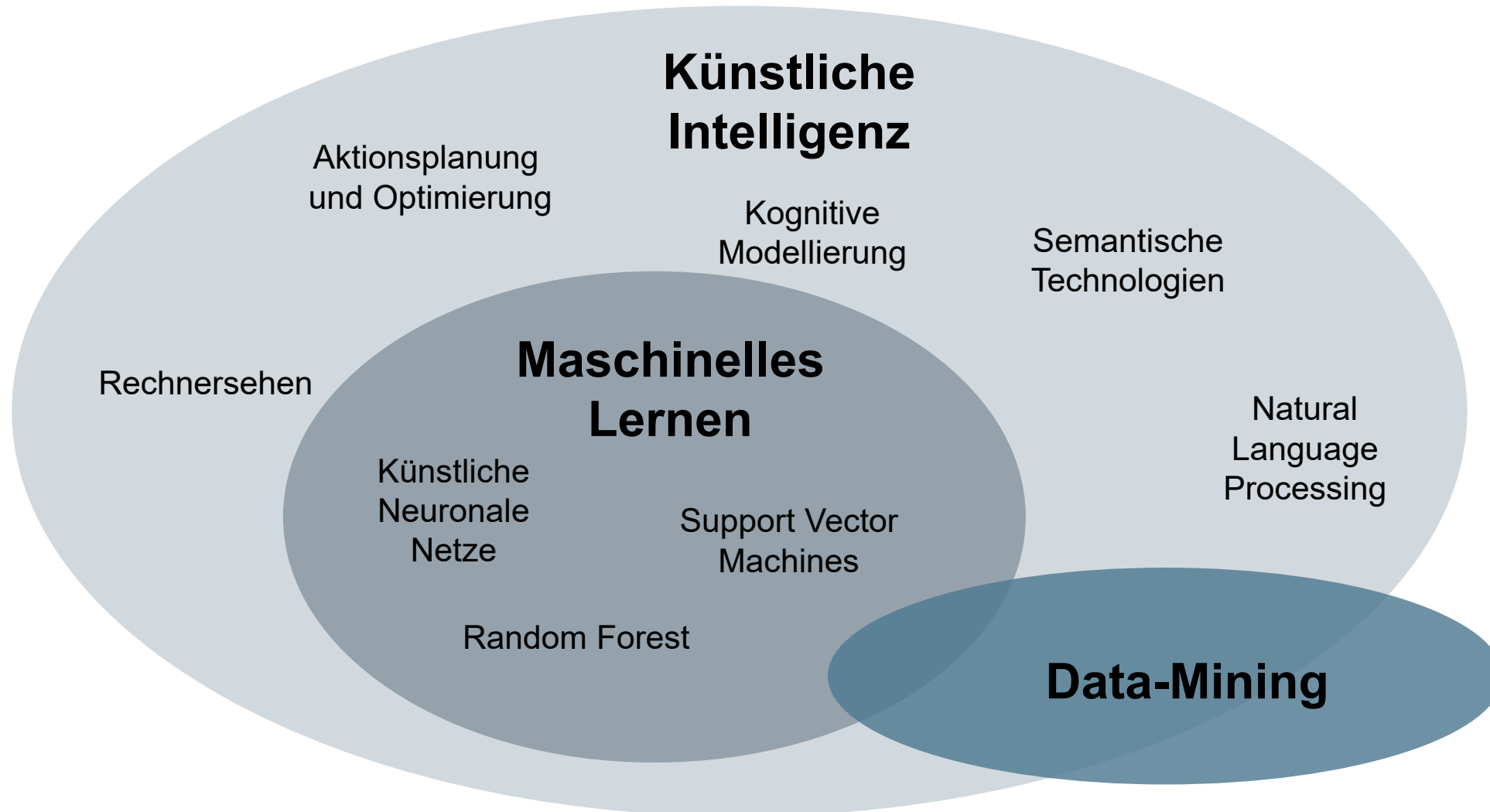
- Grundlegende Definition und Parameter
- Allgemeines Vorgehen bei der Anwendung
- Faltende neuronale Netze zur Verarbeitung von Bildern
- Rekurrente Neuronale Netze zur Verarbeitung von Zeitreihen

Anwendung in der Praxis

- Datenätze, Hardware, Softwaretools, Transferlernen



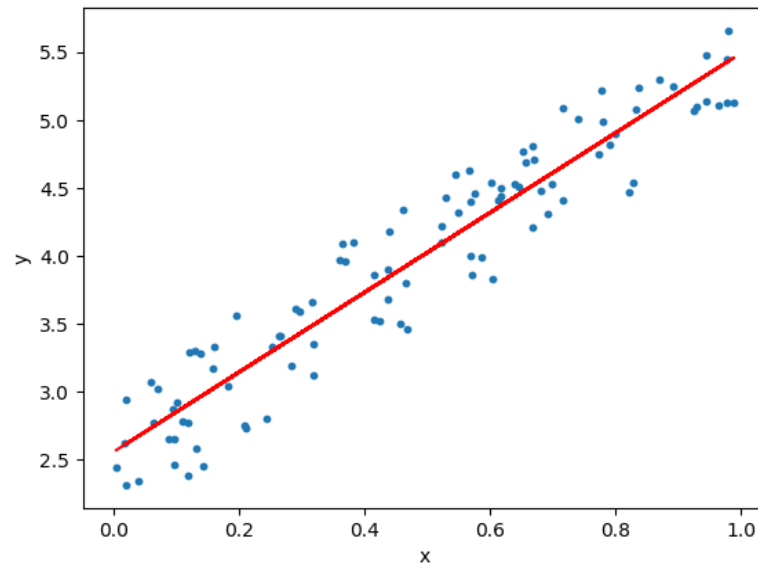
Maschinelles Lernen ist ein Überbegriff für zahlreiche Methoden und Algorithmen und ist ein Teilgebiet der Künstlichen Intelligenz.



Das Maschinelle Lernen hat das Ziel Daten quantitativ in Beziehung zu setzen oder bestimmten Klassen zuzuweisen.

Regression

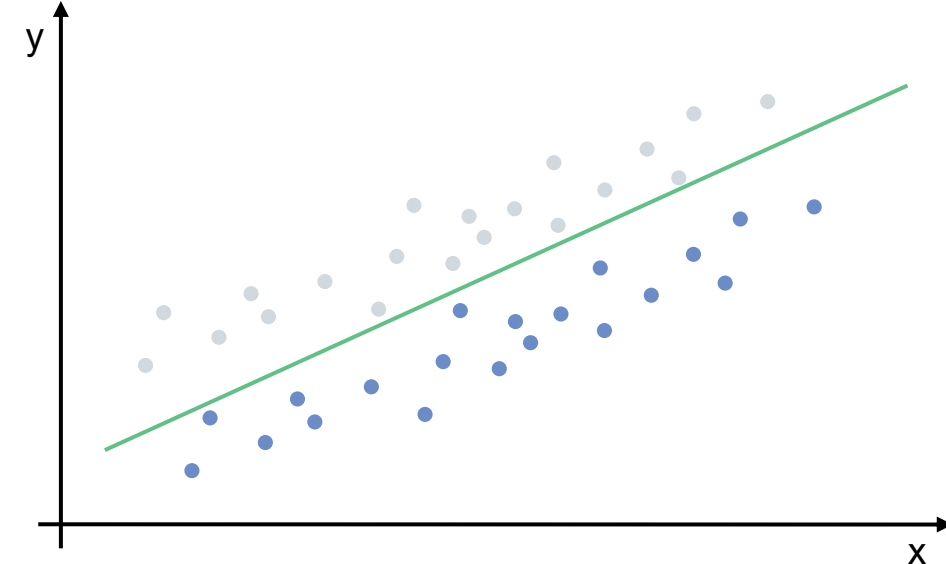
- Daten werden quantitativ in Beziehung gesetzt
- Ausgabe eines Zahlenwertes
- Beispiel: Lineare Regression
 - Erstellung eines linearen Zusammenhangs von Daten
 - Minimierung des Fehlers zwischen Gerade und Daten



Bildquelle: Medium | Towards Data Science

Klassifikation

- Daten werden einer bestimmten Klasse zugeordnet
- I.d.R. Regression mit Schwellwertbildung
- Ausgabe eines fixen Wertes
- Beispiel: Linear trennbare Klassifikation
 - Trennung zweier Klassen durch eine Gerade
 - Minimierung von Fehlklassifikationen



Maschinelles Lernen kann in verschiedene grundlegende Lernprinzipien und den dazugehörigen Daten unterteilt werden.

Maschinelle Lernprinzipien

Überwachtes Lernen

- Musterlösungen (Labels) werden dem Algorithmus zusammen mit den Trainingsdaten übergeben
- Netz versucht Trainingsdaten zu klassifizieren und gleicht seine Ergebnisse mit den zugehörigen Labels ab

Beispielhaftes Vorgehen:

- Datensatz mit Bildern von Objekten
- Pro Objekt viele unterschiedliche Bilder
- Bilder sind mit Objektbezeichnung gelabelt
- Algorithmus lernt Bilder nach der Objektbezeichnung zu betiteln

Unüberwachtes Lernen

- Algorithmus lernt Muster in den Daten ohne der Vorgabe von Zielklassen
- Reduzierung der Dimension
- Detektion von ähnlichen Bereichen in den Eingangsdaten

Beispielhaftes Vorgehen:

- Datensatz mit Bildern von Objekten
- Pro Objekt viele unterschiedliche Bilder
- Algorithmus lernt Bilder nach der ähnlichen Inhalten zu Clustern

Selbstverstärkendes Lernen

- Training durch Belohnung und Bestrafung
- Jede Lösung wird mit Punktezahl bewertet, die es zu maximieren gilt

Beispielhaftes Vorgehen:

- Algorithmus bekommt ein Sensorinput
- Zufällige Verarbeitung des Inputs und Generierung eines Outputs
- Output wird entsprechend eines Schemas bewertet
- Verstärkung des Algorithmus bei erfolgreichem Output

Zu den etablierten Algorithmen des Maschinellen Lernens zählen Support Vector Machines, Random Forests und Artificial Neural Networks.

Support Vector Machine

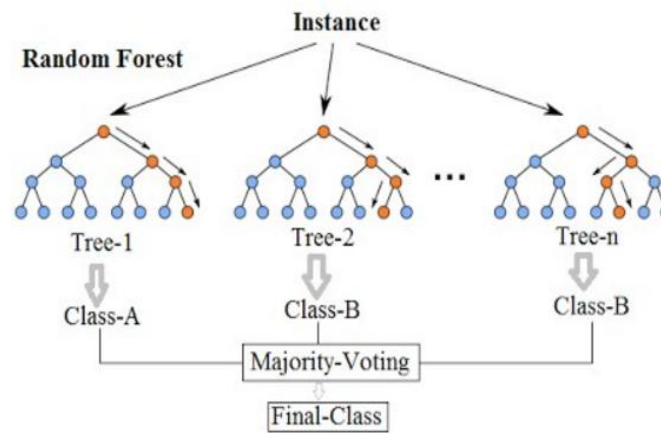
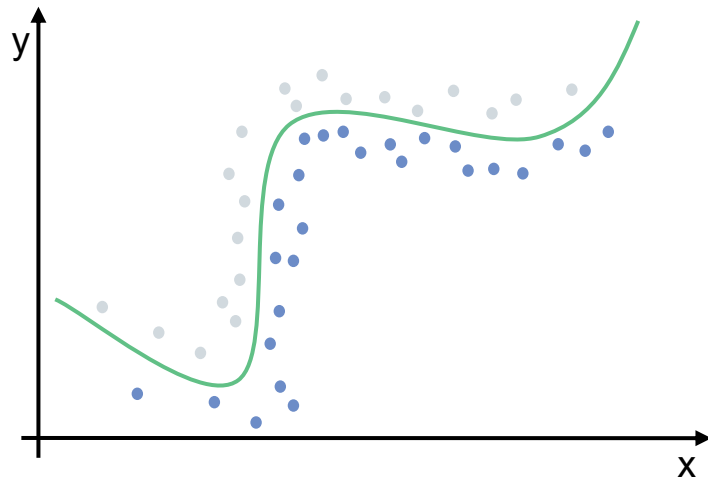
- Klassifikation oder Regression von nicht linear trennbaren Daten
- Repräsentation der Daten als Vektoren in Vektorraum
- Einführung von Hyperebenen zur linearen Trennung
- Überführung der Daten in einen höherdimensionalen Raum zur linearen Klassifikation

Random Forests

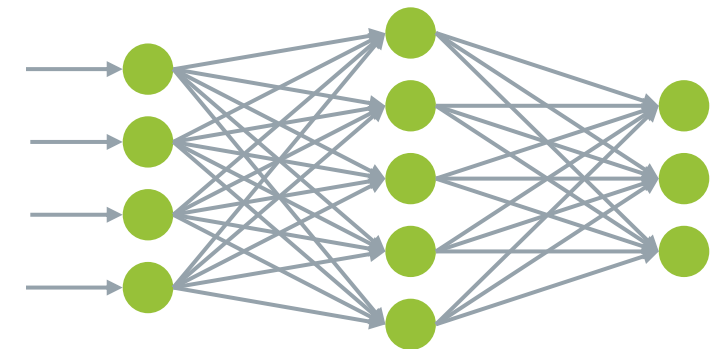
- Klassifikation oder Regression von Daten durch unkorrelierte Entscheidungsbäume
- Wahl eines Parameters und Treffen der Entscheidung auf Basis des Wertes
- Jeder Entscheidungsbaum gibt eine Klasse aus
- Auswahl der endgültigen Klasse auf Basis der häufigsten Klasse

Artificial Neural Networks

- Klassifikation oder Regression von Daten durch eine Vielzahl von künstlichen Neuronen
- Neuron multipliziert Eingangswert mit Gewichtungsfaktor, Propagierungsfunktion und Aktivierungsfunktion
- Vielfältige und komplexe Architekturen möglich



Bildquelle: Medium



Gliederung VE10

„Maschinelles Lernen“

Motivation und Überblick

Grundlagen Maschinelles Lernen

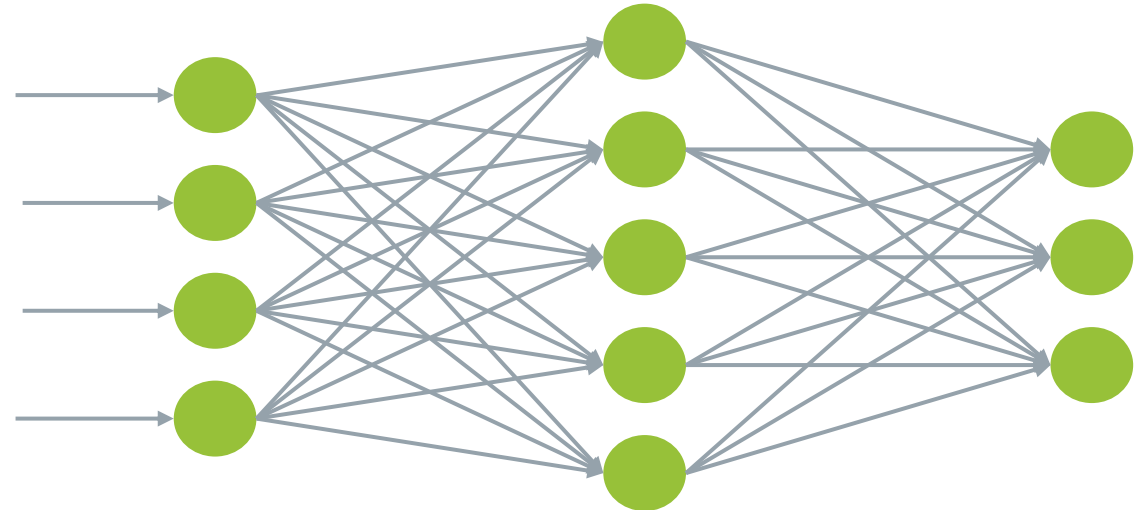
- Grundprinzipien und Ziele des maschinellen Lernens
- Grundlegende etablierte Algorithmen des maschinellen Lernens

Künstliche Neuronale Netze

- Grundlegende Definition und Parameter
- Allgemeines Vorgehen bei der Anwendung
- Faltende neuronale Netze zur Verarbeitung von Bildern
- Rekurrente Neuronale Netze zur Verarbeitung von Zeitreihen

Anwendung in der Praxis

- Datenätze, Hardware, Softwaretools, Transferlernen



Mittels künstlicher neuronaler Netze konnte 2012 ein Durchbruch bei der Klassifikation von Bildern erreicht werden

Large Scale Visual Recognition Challenge:

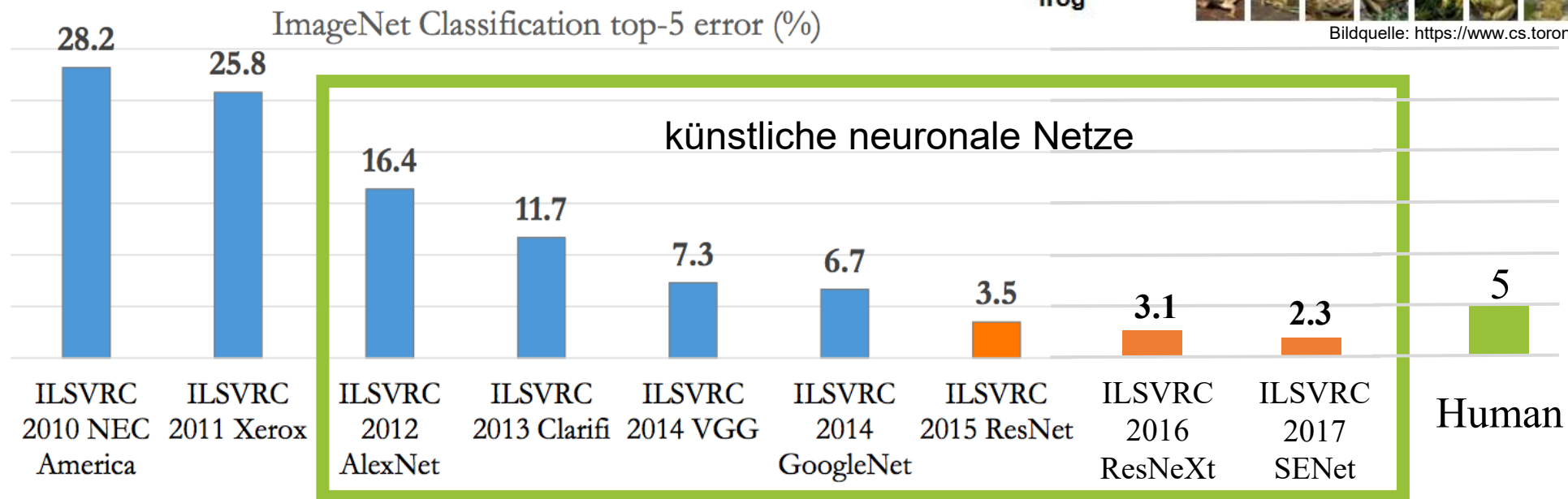
- Jährlicher Wettbewerb zur Klassifizierung von 1000 verschiedenen Objekten aus 150.000 Fotos
- ImageNet Datenbank (14 Millionen Fotos, 21.000 semantische Bezeichnungen)

Ergebnisse der letzten Jahre:

airplane
automobile
bird
cat
deer
dog
frog

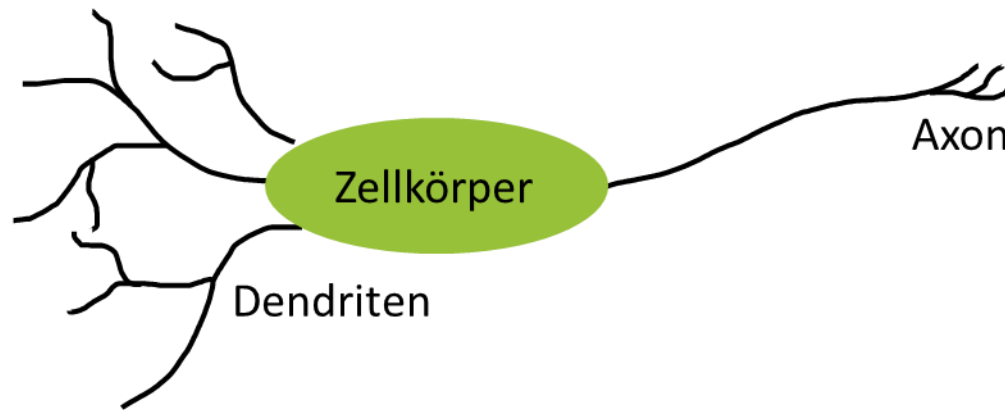


Bildquelle: <https://www.cs.toronto.edu/~kriz/cifar.html>



Ein künstliches Neuron ist die Nachbildung eines biologischen Neurons unter Verwendung mathematischer Funktionen.

Biologisches Neuron



Dendriten

- Verästelungen zum Sammeln der Signale aus verschiedenen Neuronen

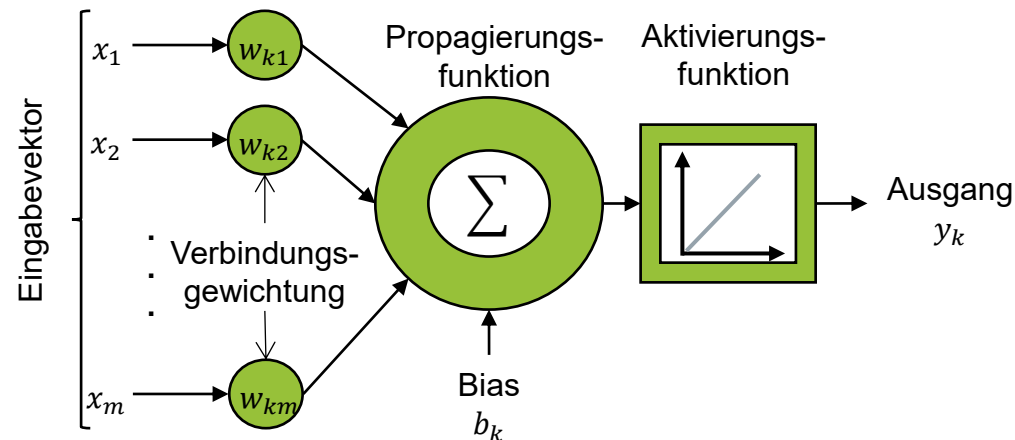
Zellkörper

- Aggregation der Signale und Erzeugung eines Aktionspotentials

Axon

- Weiterleitung des Signals an weitere Neuronen

Künstliches Neuron (Perzeptron)



Propagierungsfunktion

- Fusionierung der Eingangswerte (z.B. Summe, Produkt, Max, Min)

Aktivierungsfunktion

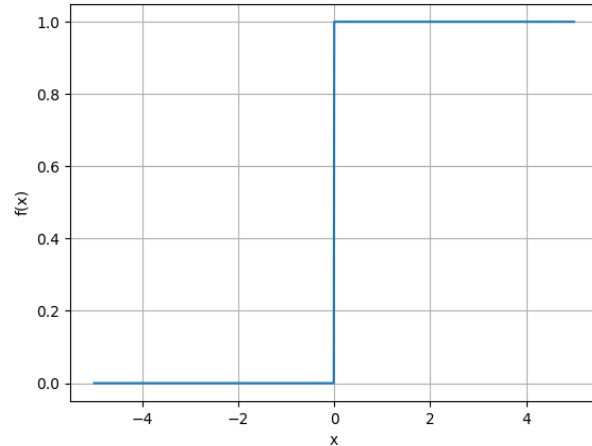
- Funktion zur Berechnung des Ausgangssignals

Bias

- Parameter für Aktivierung des Neurons

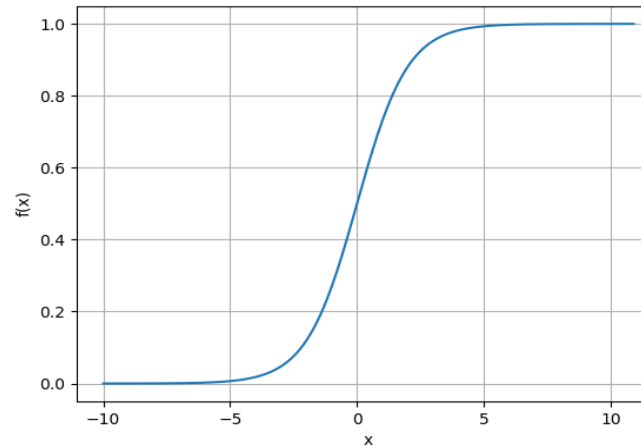
Je nach Anwendung haben sich verschiedene Aktivierungsfunktionen etabliert, dabei wird die Aktivierungsfunktion Rectified Linear Unit jedoch am häufigsten eingesetzt.

Sprungfunktion



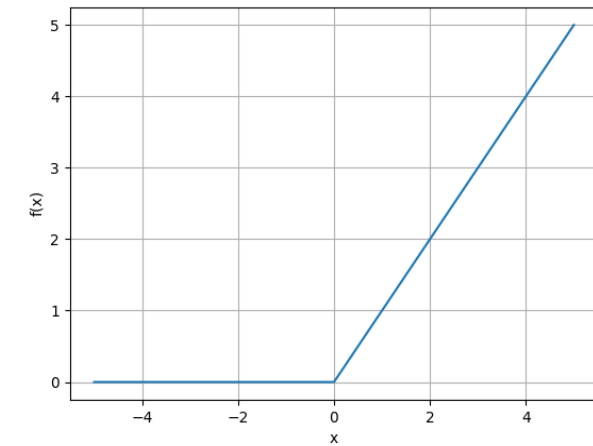
$$y(h) = \begin{cases} 1 & h > \theta \\ 0 & h \leq \theta \end{cases}$$

Sigmoidfunktion



$$y(h) = \text{sig}(h) = \frac{1}{1 + e^{-h}}$$

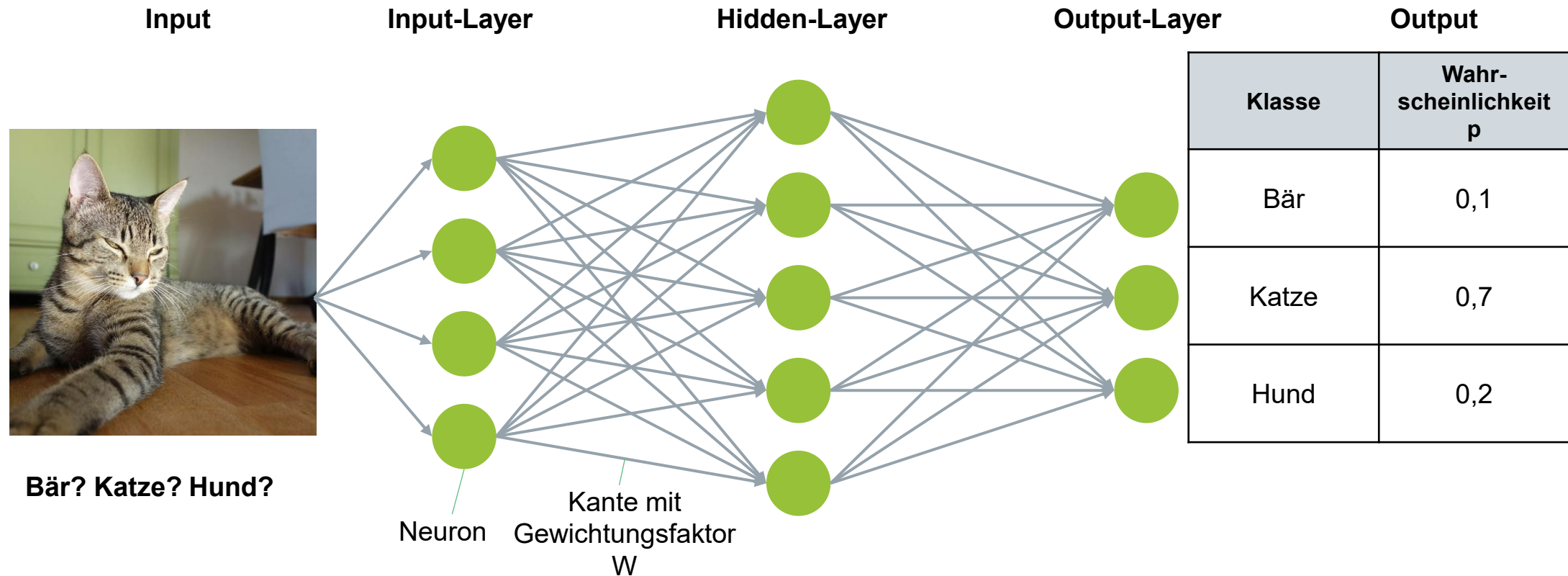
Rectified Linear Unit (ReLu)



$$y(h) = \max(0, h) = \begin{cases} 0 & h \leq 0 \\ h & h > 0 \end{cases}$$

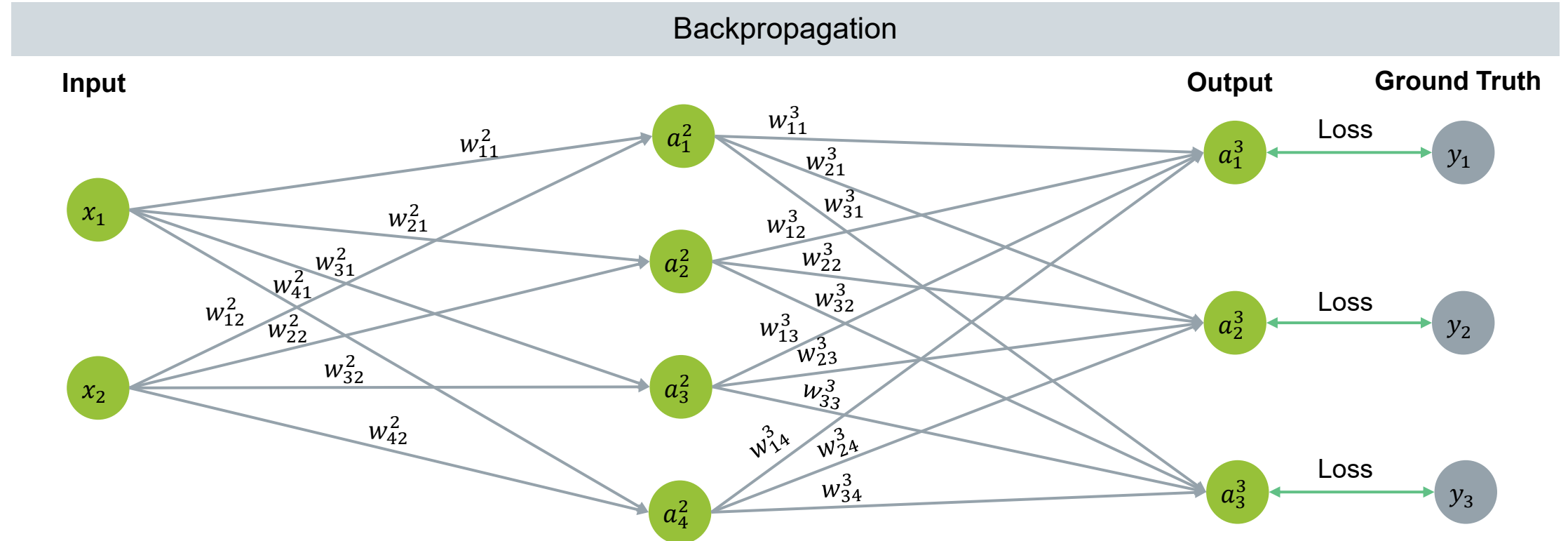
Die Struktur künstlicher neuronaler Netze basiert auf Schichten mehrerer Neuronen, die über Kanten miteinander verknüpft sind.

Aufbau eines künstlichen neuronalen Netzes (KNN)



■ Wird in der Literatur als Multi-Layer Perceptron (MLP) bezeichnet

Das Lernen des künstlichen neuronalen Netzes erfolgt durch Anpassung der Kantengewichte im Trainingsprozess.



- Berechnung des Fehlers (Verlust/Loss) des Outputs
- Berechnung des Gradienten der Fehlerfunktion vom Output bis zum Input
- Anpassung der Gewichte in negativer Gradientenrichtung mit bestimmter Schrittweite

Über verschiedene Verlustfunktionen kann das Lernverhalten optimiert werden.

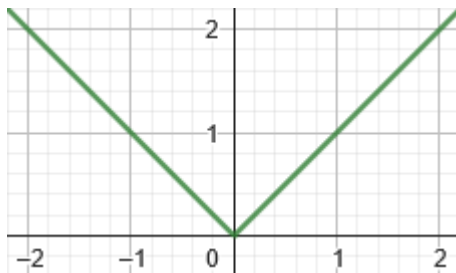
Vorhanden Variablen/Daten

- Anzahl der Trainingsdaten N
- Anzahl der Output Neuronen K
- Korrektes Ergebnis, Label $t_{n,k}$
- Tatsächlicher Output $y_{n,k}$

Verlustfunktionen

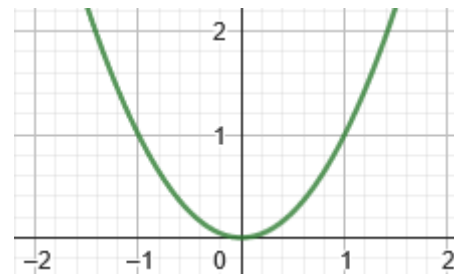
L_1 Loss

$$L_1 = \sum_{n=1}^N \sum_{k=1}^K |t_{n,k} - y_{n,k}|$$



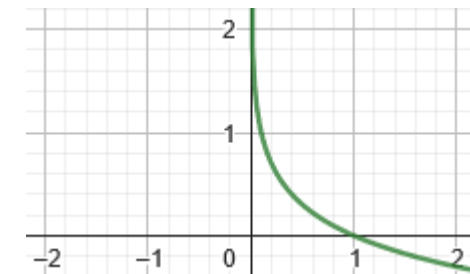
L_2 Loss

$$L_2 = \sum_{n=1}^N \sum_{k=1}^K (t_{n,k} - y_{n,k})^2$$

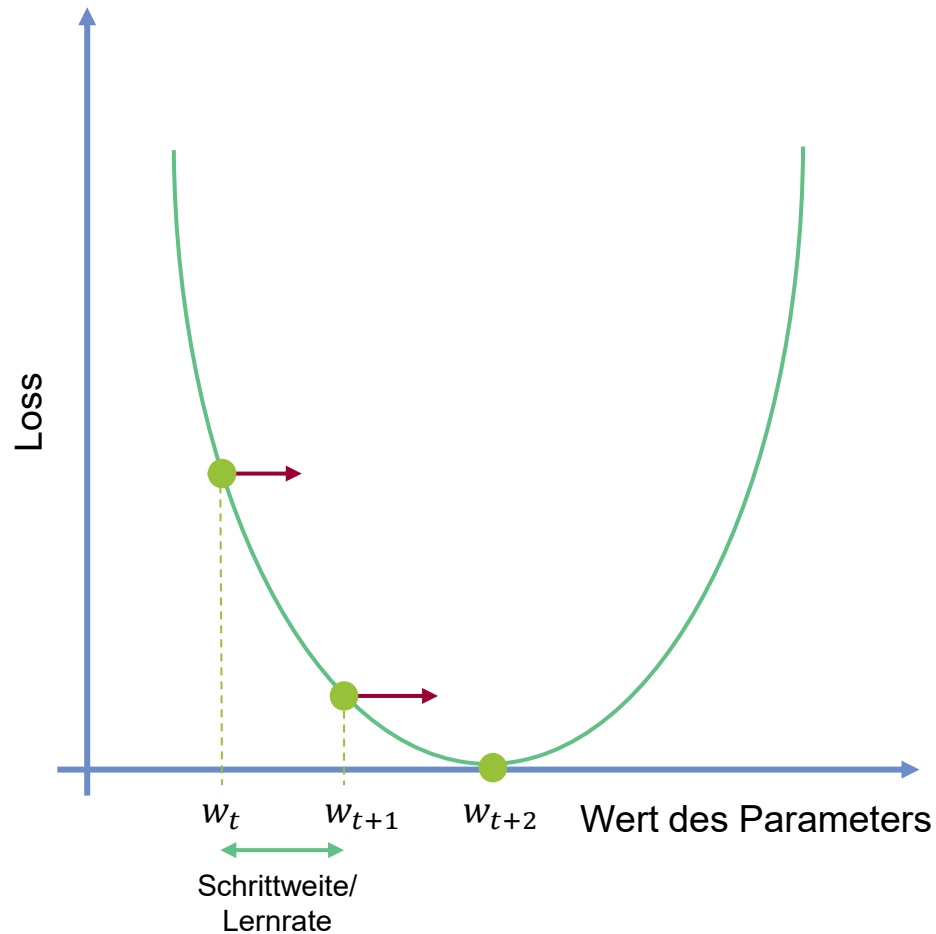


Cross-Entropy Loss

$$L_{CE} = \sum_{n=1}^N \sum_{k=1}^K -t_{n,k} \log(y_{n,k})$$



Das Gradientenabstiegsverfahren ist ein grundlegendes Lernverfahren bei künstlichen neuronalen Netzen, dessen Lernrate über den Erfolg des Lernens entscheidet.

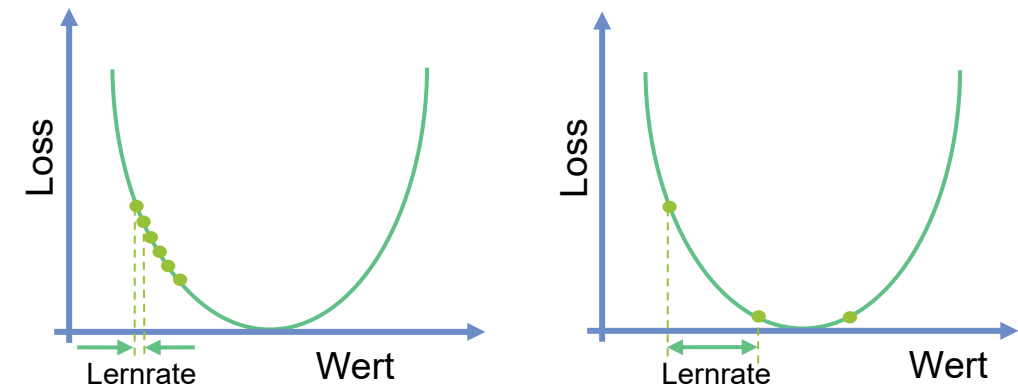


Vorgehen

- Berechnung des Loss mit initialem Wert des Parameters w_t
- Berechnung des Gradienten und der negativen Richtung
- Anpassung des Parameters mit definierter Schrittweite
- Iteratives Vorgehen bis Loss minimal wird
- Auch Stochastic Gradient Descent (SGD) Verfahren genannt, da Daten zufällig gewählt werden

Lernrate entscheidet über Erfolg des Lernprozesses

- Zu klein → Ineffizient, Festsitzen in lokalem Minimum
- Zu groß → Oszillieren um Minimum



Im Laufe der Zeit haben sich zahlreiche Erweiterungen des Gradientenabstiegsverfahrens etabliert, die den jeweiligen Nachteilen begegnen sollen.

Mini-Batch Stochastic Gradient Descent (SGD)

- Berechnung des Gradienten auf Basis einer bestimmten Anzahl an zufällig ausgewählten Daten (Mini-Batch)

SGD mit Nesterov Momentum

- SGD führt häufig zu einem Oszillieren um das Minimum
- Nesterov Momentum vergrößert die Lernrate bei konstanter Richtung und verkleinert diese bei Richtungsänderungen, um das Oszillieren zu vermeiden

Adaptive Gradient (AdaGrad)

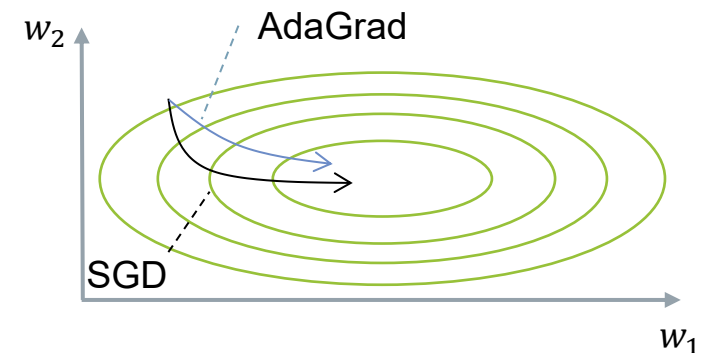
- Division der Lernrate durch die Wurzel der Summe aller Gradienten zum Quadrat
- Kleiner Gradient $\rightarrow$ Große Lernrate
- Großer Gradient $\rightarrow$ Kleine Lernrate

Root Mean Square Propagation (RMSProp)

- AdaGrad Lernrate konvergiert durch die Division nach einigen Iterationsschritten zu Null
- RMSProp dividiert hierbei durch laufenden quadratischen Mittelwert, um die ungewollte Konvergenz zu vermeiden

Adaptive Momentum Estimation (Adam)

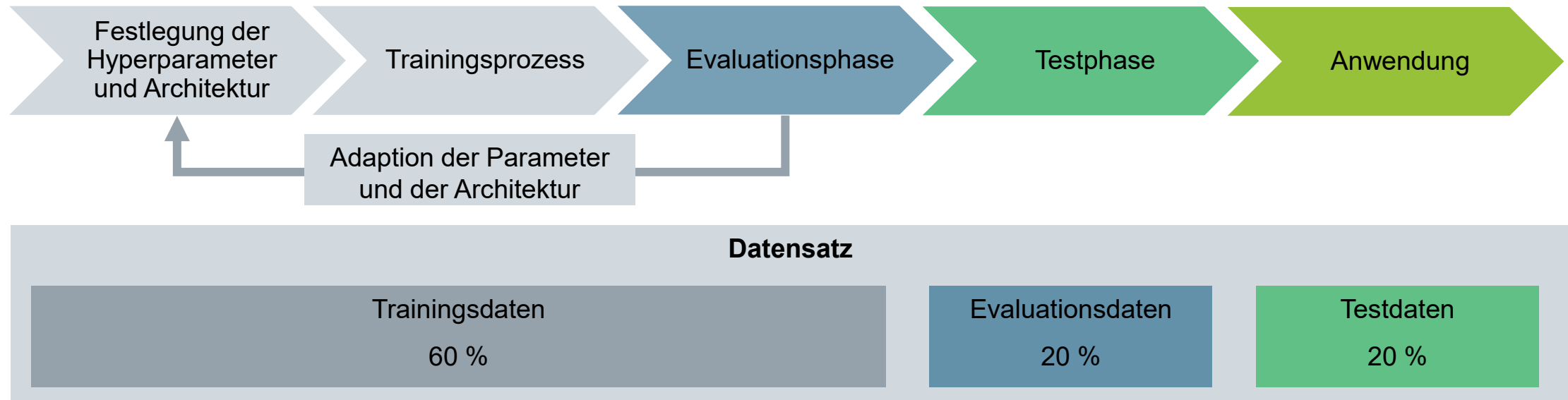
- Kombination von Momentum und RMSProp unter Berücksichtigung des Impulses der zweiten Ordnung des Gradienten (Varianz)
- Effizientere und robustere Auswahl der Lernrate



Quelle: <https://towardsdatascience.com/neural-network-optimization-7ca72d4db3e0>, <https://algorithmia.com/blog/introduction-to-optimizers>

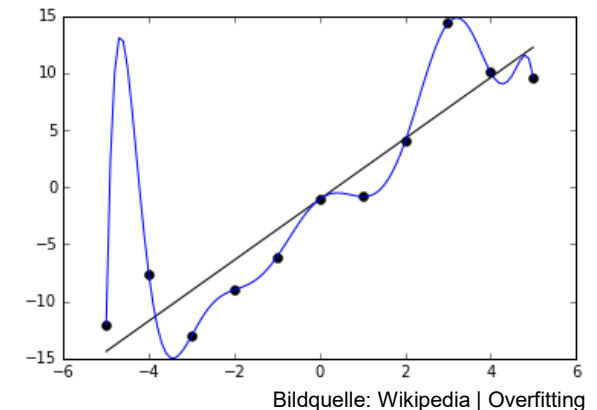
Ein umfangreicher Datensatz bildet die entscheidende Grundlage für eine erfolgreiche Anwendung von künstlichen neuronalen Netzen.

Iteratives Vorgehen zur Entwicklung eines künstlichen neuronalen Netzes

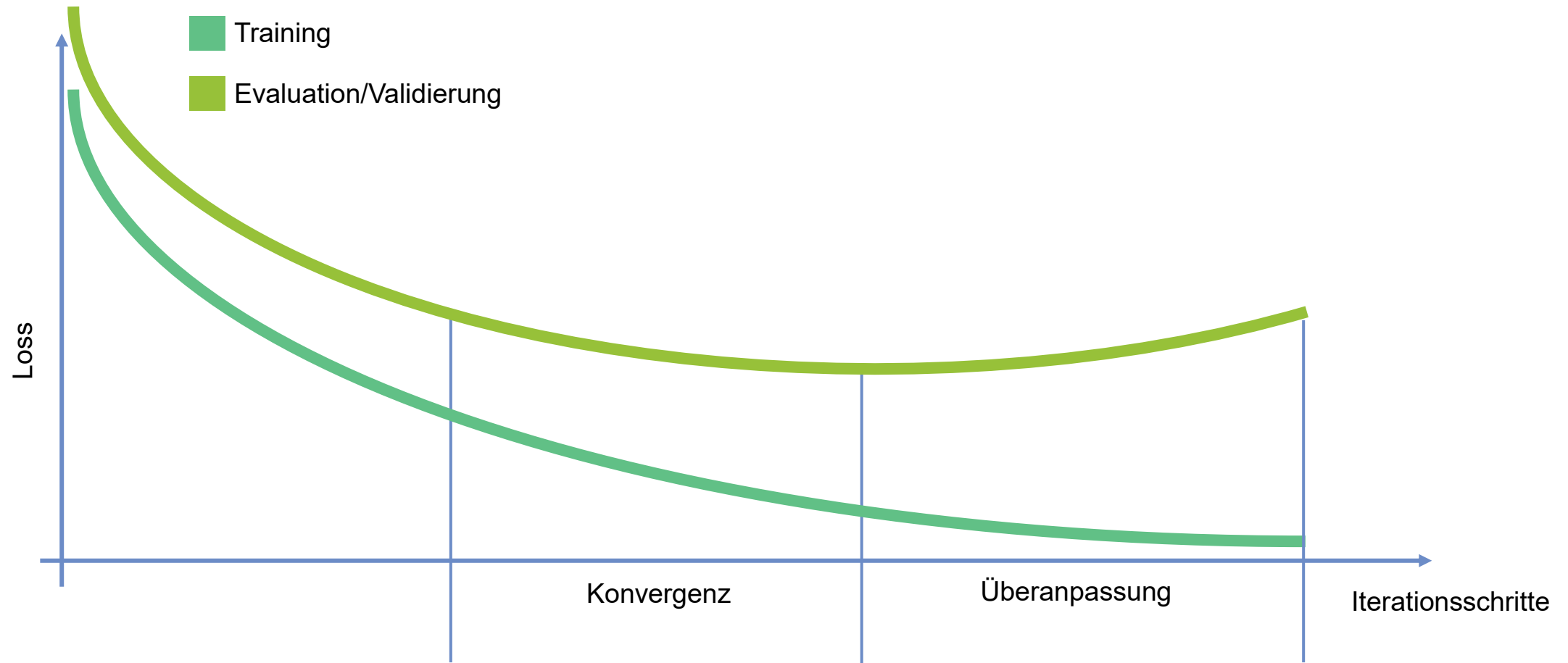


Anforderungen an den Datensatz

- Korrekte Klassifikation (Supervised Learning)
- Strikte Trennung zwischen Trainings-, Evaluation- und Testdaten
- Hohe Varianz der möglichen Szenarien sollten im Datensatz abgedeckt sein
- Ausreichende Menge an Daten
- Daten und Klassen sollten dem Programm leicht zur Verfügung gestellt werden können



Das Ziel des Trainings ist die Minimierung des Losses über mehrere Iterationsschritte hinweg.



Überanpassung (engl. Overfitting)

- Modell passt sich zu sehr an Trainingsdaten an und ist nicht in der Lage das Gelernte zu generalisieren

Zur Vermeidung von Überanpassung haben sich unterschiedliche Methoden etabliert.

Methoden zur Vermeidung von Überanpassung

■ Reduktion der Größe des Künstlichen Neuronalen Netzes

- Das Netz lernt eher allgemeine Inhalte als spezifische Inhalte einzelner Daten

■ Frühzeitiges Beenden des Lernprozesses

■ L_1 Regularisierung

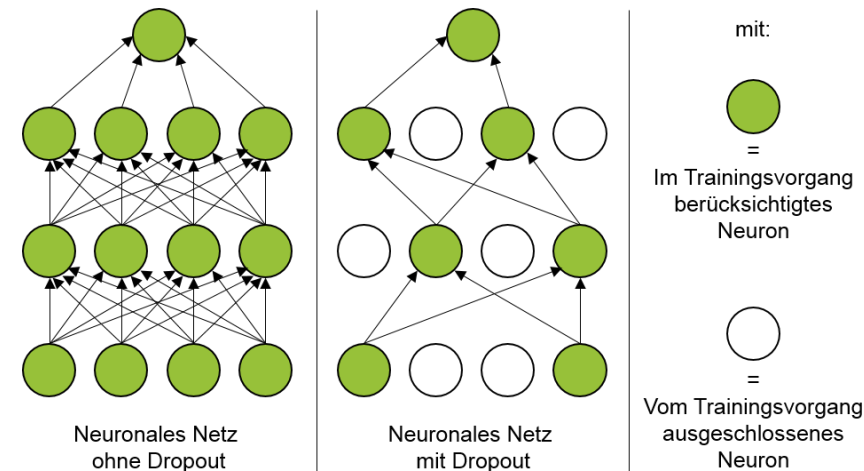
- Betrag der Parameter tragen zur Kostenfunktion bei → größere Parameter werden bestraft und erhöhen den Loss
- Parameter mit geringen Gewichten im Vergleich zum Durchschnitt werden auf 0 gesetzt

■ L_2 Regularisierung

- Summe der quadrierten Parameter tragen zur Kostenfunktion bei → umso größere Parameter werden verstärkt bestraft und erhöhen den Loss
- Parameter mit geringen Gewichten im Vergleich zum Durchschnitt werden nahe 0 gebracht

■ Dropout Regularisierung

- Bestimmter Anteil der Neuronen wird abwechselnd, zufällig abgeschaltet
- Netz lernt Daten mit verbleibenden Neuronen
- Üblicherweise Anteil der deaktivierten Neuronen zwischen 30% und 60%

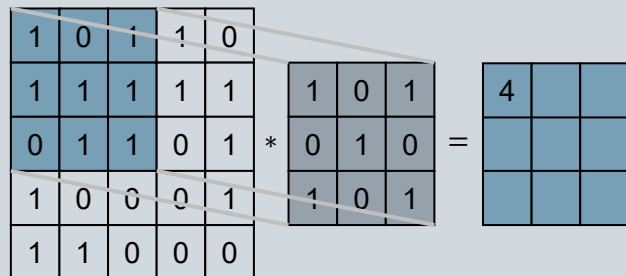


Für die Bildverarbeitung mittels künstlicher neuronaler Netze haben spezielle Netzschichten etabliert.

Zusammensetzung verschiedener Schichten ergeben ein Convolutional Neural Network

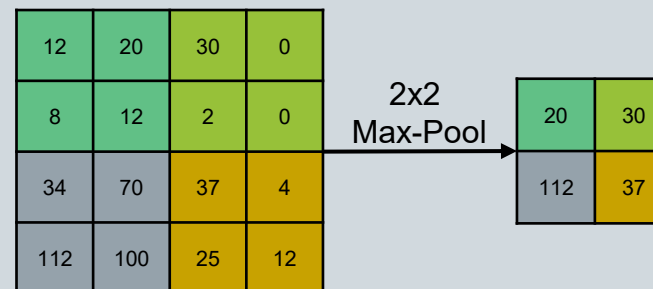
Faltungsschichten (Convolutional Layers)

- Datenverarbeitung hauptsächlich in Faltungsschichten
- Filter werden mit den Eingabedaten mathematisch gefaltet
- Ergebnis ist eine „Activation-Map“ (auch: Feature-Map)



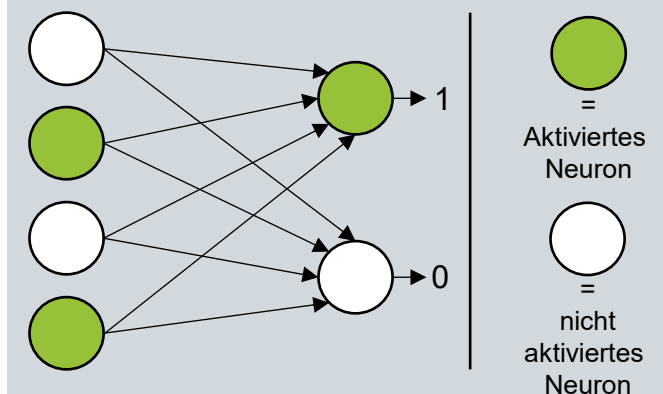
Aggregationsschichten (Max Pooling Layer)

- Dienen der Verkleinerung der Daten
- Verringerung des Verarbeitungsaufwands der Folgeschichten
- Aus einem Feld wird der größte (dominanteste) Wert ausgewählt

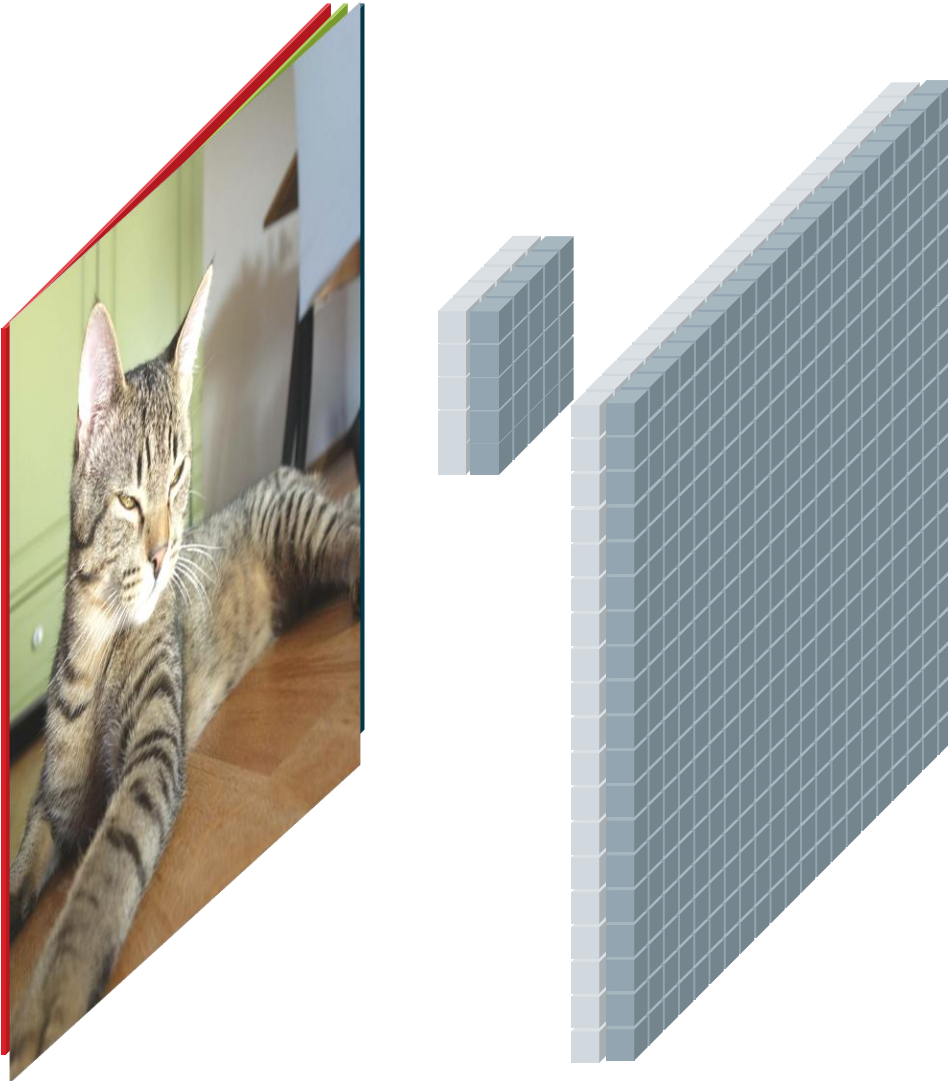


Vollvernetzte Schichten (Fully Connected Layers)

- Verknüpfung mit allen Neuronen der vorherigen Schicht
- Zusammenfassung der Ergebnisse der vorherigen Schichten
- Z.B. Output Layer: Jedes Neuron repräsentiert eine mögliche Klasse



Faltungsschichten wenden mehrere Faltungsoperatoren auf das Eingangsbild an und berechnen so Merkmalskarten.



Beispiel einer Faltungsschicht

- Größe des Eingangsbildes: $250 \times 250 \times 3$
- Größe des Faltungsoperators: $5 \times 5 \times 3$
 - Pro Kanal (Tiefe) im Eingang ein Faltungsoperator
- Größe der Merkmalskarte: $250 \times 250 \times 1$
 - Ergebnisse der 3 Faltungsoperatoren werden pro Pixel zu einem Wert addiert
- Verwendung von insgesamt 2 Filtern (= Tiefe der Faltungsschicht)
- Größe der zweiten Schicht $250 \times 250 \times 2$
- In der Praxis Kombination aus mehreren Faltungs-, Pooling- und vollvernetzten Schichten

Lernvorgang

- Identische Lernalgorithmen wie beschrieben
- Anpassung der Matrixelemente der Filteroperatoren

Das Verhalten am Rand der Eingangsmatrix und die Schrittweite beeinflussen die Größe der Ergebnismatrix.

Padding

1	0	1	1	0
1	1	1	1	1
0	1	1	0	1
1	0	0	0	1
1	1	0	0	0

→

0	0	0	0	0	0	0
0	1	0	1	1	0	0
0	1	1	1	1	1	0
0	0	1	1	0	1	0
0	1	0	0	0	1	0
0	1	1	0	0	0	0
0	0	0	0	0	0	0

Vorgehen am Rand des Eingangsbildes

- Vergrößerung der Eingangsdaten durch Nullen, Zufallszahlen, Spiegelung
- Vergrößerung um die Hälfte der Breite/Höhe der Filtermatrix
- Vermeidet die Verkleinerung der Merkmalskarte

Striding

Schritt 1

1	0	1	1	0
1	1	1	1	1
0	1	1	0	1
1	0	0	0	1
1	1	0	0	0

Schritt 2

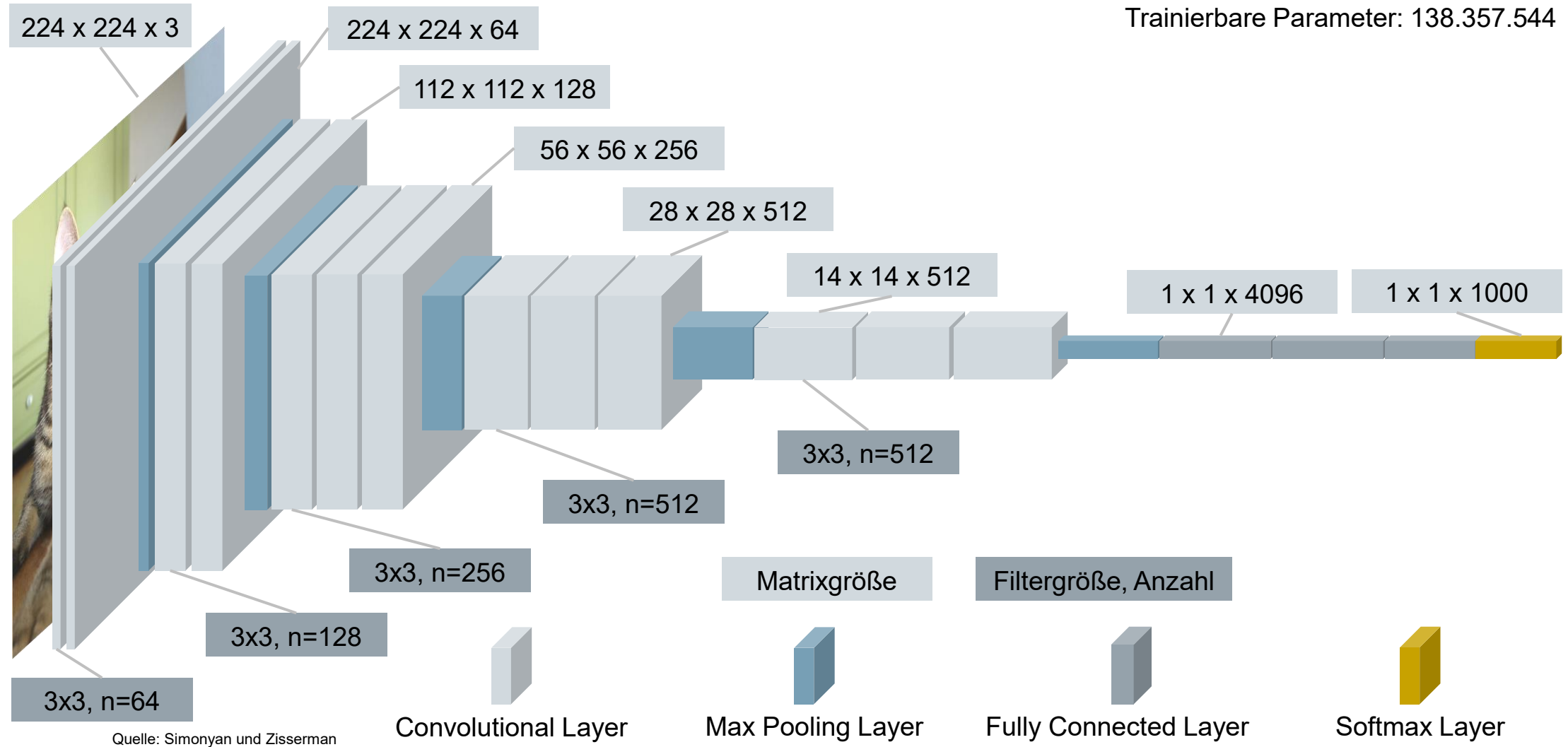
1	0	1	1	0
1	1	1	1	1
0	1	1	0	1
1	0	0	0	1
1	1	0	0	0

Schrittweite

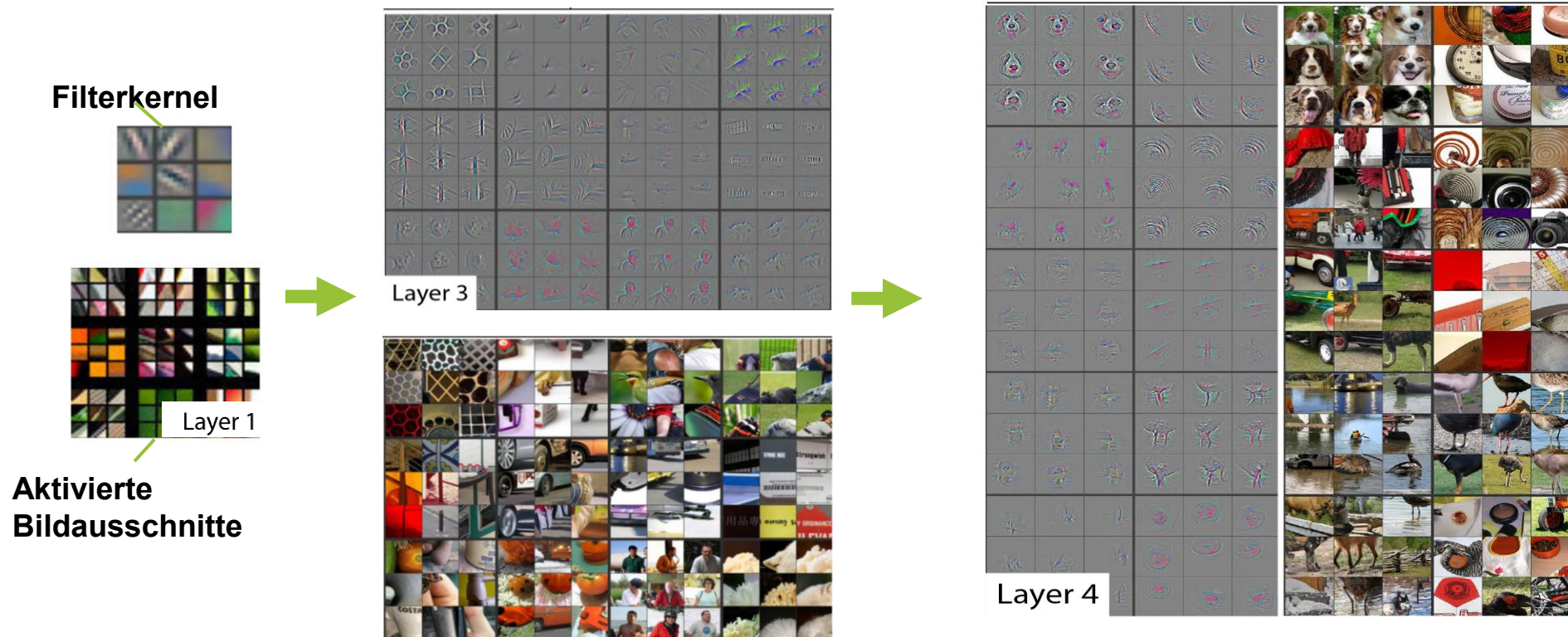
- Verschieben der Filtermatrix nach jeder Faltungsoperation
- Schrittweite wird in jede Richtung angegeben
- Schrittweiten größer 1 verkleinern die Merkmalskarte

<https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529>

Hochfunktionale Convolutional Neural Networks wie die VGG16-Architektur sind aus zahlreichen repetitiven Schichten aufgebaut.



Trainierte Faltende Neuronale Netze aktivieren je nach Schicht unterschiedliche Merkmale in den Eingangsbildern.



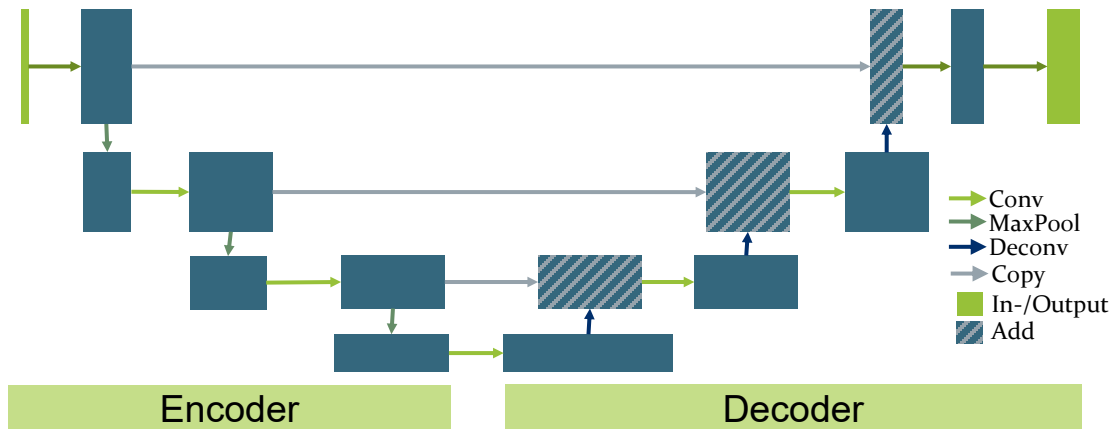
Bildquelle: Zeiler and Fergus. Visualizing and understanding convolutional networks. 2014

Beobachtung

- In frühen Schichten werden kleine Merkmale (z.B. Kanten, Ecken) aktiviert
- Spätere Schichten aktivieren komplexere Merkmale bis hin zu charakteristischen Merkmalen der zu detektierenden Objekte

Zur detaillierten Interpretation von Bildern erfolgt mittels Encoder-Decoder-Architekturen eine pixelbasierte, semantische Segmentierung.

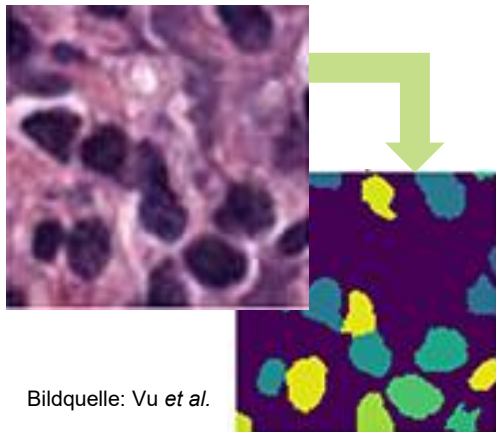
Semantische Segmentierung von Farbbildern



Encoder-Decoder-Architekturen

- Eingangsbilder werden im Encoder verarbeitet
- Merkmalskarten werden im Decoder wieder entfaltet und auf ursprüngliche Größe vergrößert
- Zusätzlich werden Merkmalskarten der gleichen Größe aus Encoder und Decoder addiert (UNet-Architektur)
- Resultierendes Bild weist jedem Pixel einen Wert zu, der einer semantischen Klasse entspricht

Histopathologie

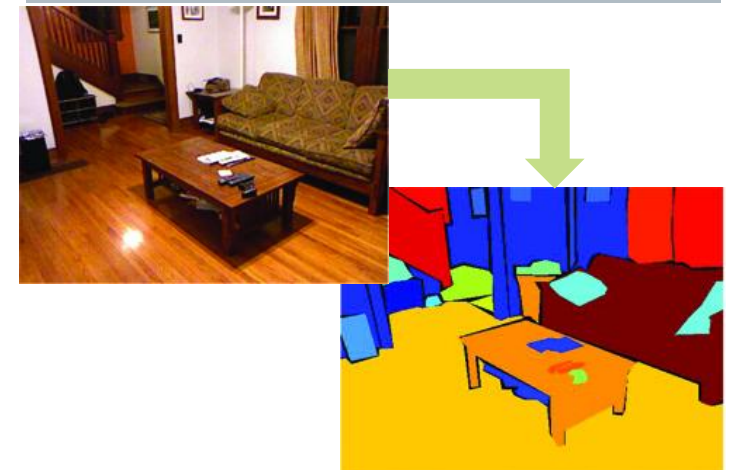


Autonomes Fahren



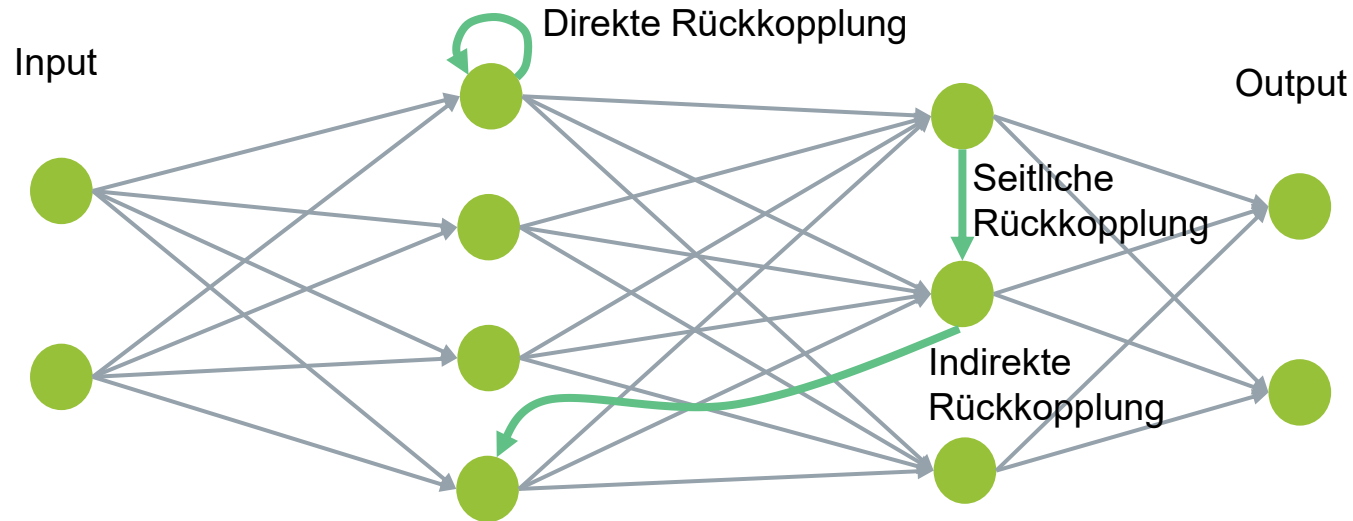
Bildquelle: <https://www.cityscapes-dataset.com/>

Servicerobotik



Rekurrente neuronale Netze ermöglichen die Verarbeitung von zeitlich abhängigen Datenreihen.

Rückkopplungsarten von Rekurrenten Neuronalen Netzen (RNN)



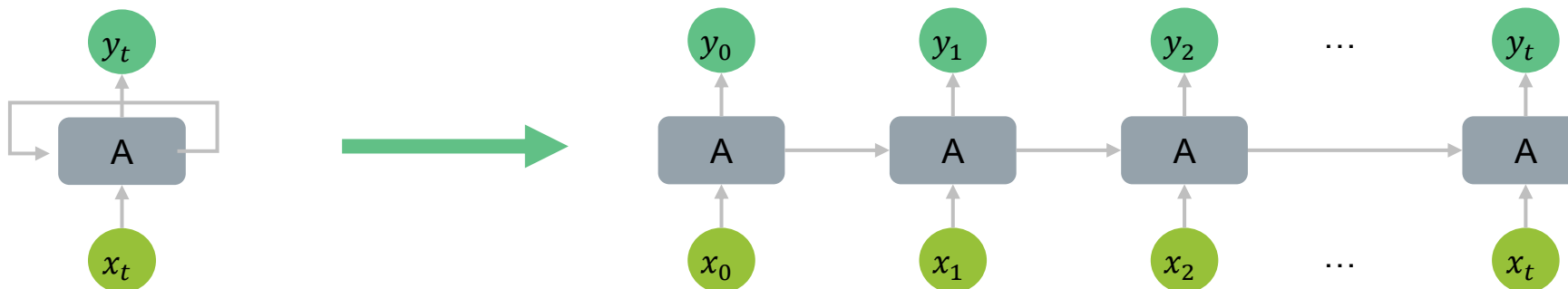
Eigenschaften von RNN

- Neuronen sind nicht nur mit dem nach folgenden Neuronen verknüpft
- Direkte und Indirekte rekurrente Neuronen können als Speicher dienen
- Die häufigste Rückkopplungsart ist die direkte Rückkopplung

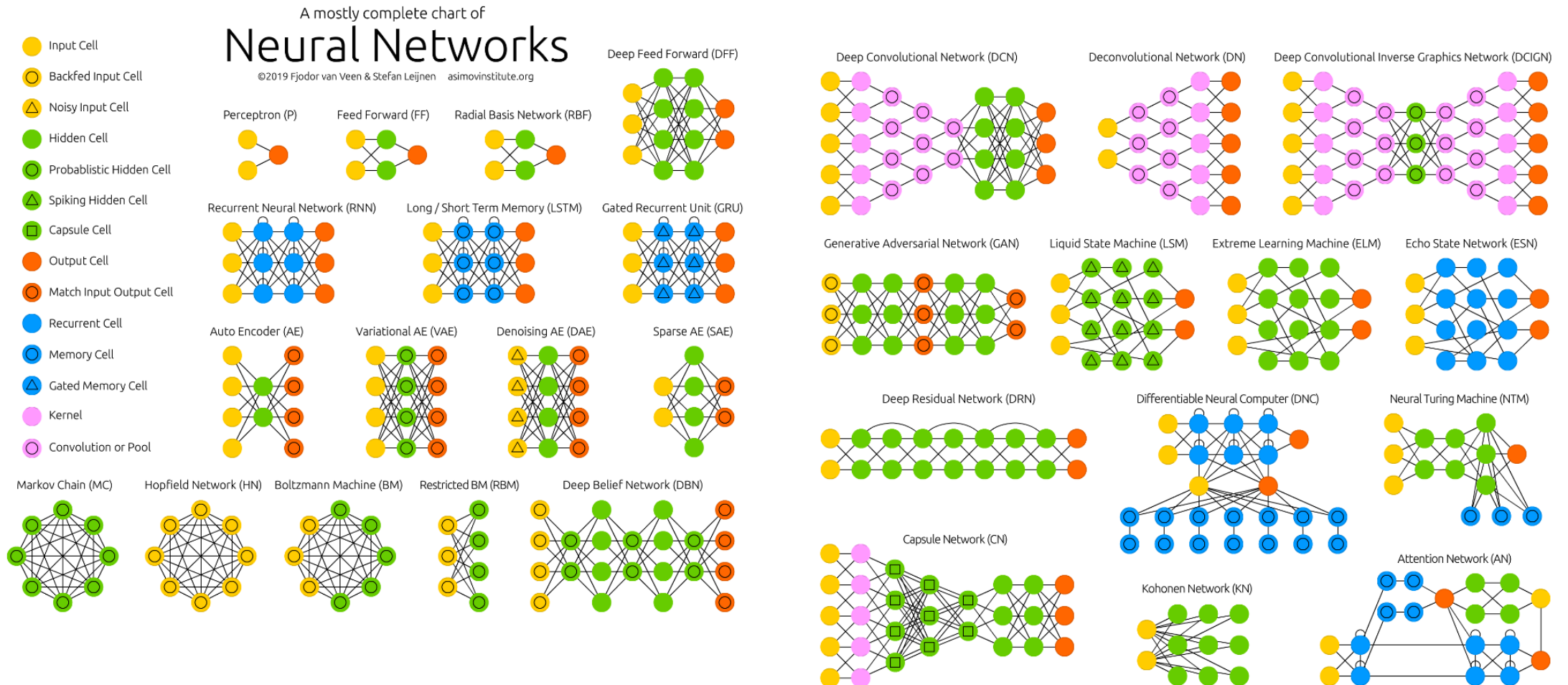
Anwendungen

- Verarbeitung von Daten als Zeitreihen
- Z.B. Sprachverarbeitung

Direkte Rückkopplung

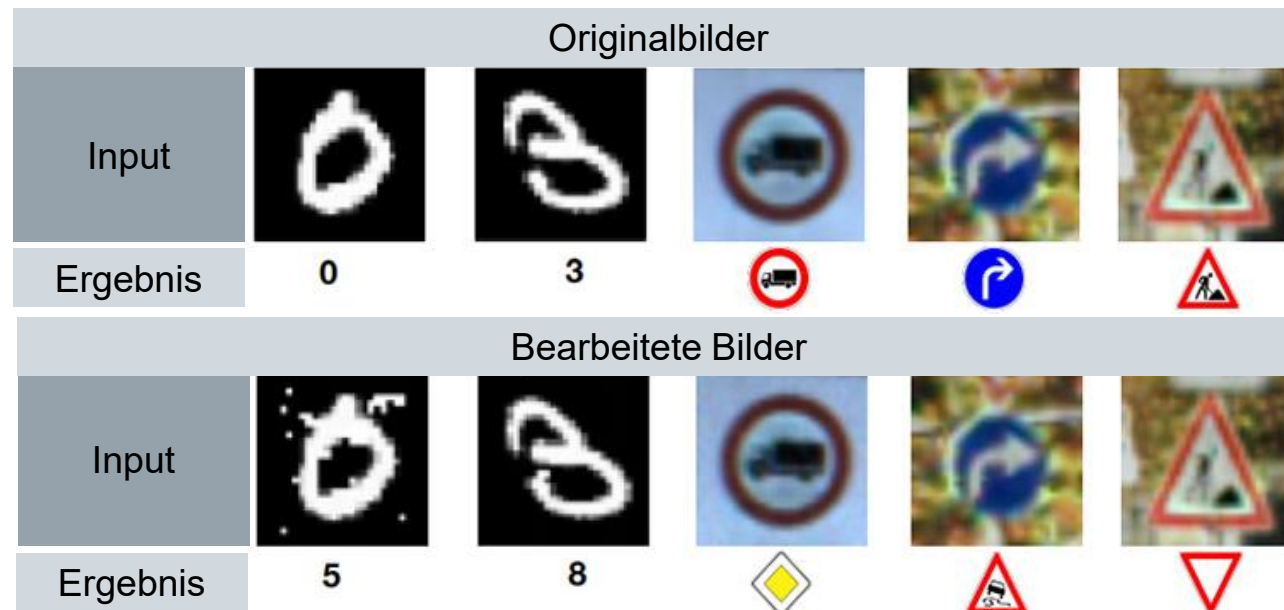
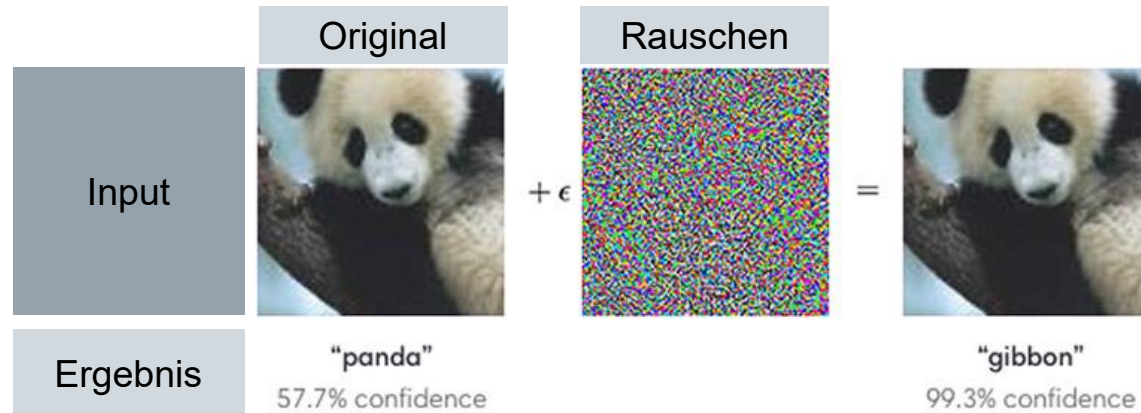


Neuronale Netze können in vielfältige grundlegende Architekturen untergliedert werden.



Bildquelle: <https://www.asimovinstitute.org/neural-network-zoo/>

Die Anwendung von neuronalen Netzen in sicherheitsrelevanten Anwendungen ist mit einigen Hürden verbunden.



KNN als Blackbox

- Eingeschränkte Nachvollziehbarkeit der Klassifizierung → Blackbox
- Bereits geringfügige Veränderungen der Eingangsdaten führen zu Fehlklassifikationen
- Generative Adversarial Neural Networks
 - KNN können selbst für die Erstellung von veränderten Trainingsdaten genutzt werden
 - Trainingsdaten werden für Steigerung der Robustheit genutzt
- Ian Goodfellow: „On the applied side, no one has yet designed a truly powerful defense algorithm that can resist a wide variety of adversarial example attack algorithms”

Quelle: <https://spectrum.ieee.org/cars-that-think/transportation/sensors/slight-street-sign-modifications-can-fool-machine-learning-algorithms>

Gliederung VE10

„Maschinelles Lernen“

Motivation und Überblick

Grundlagen Maschinelles Lernen

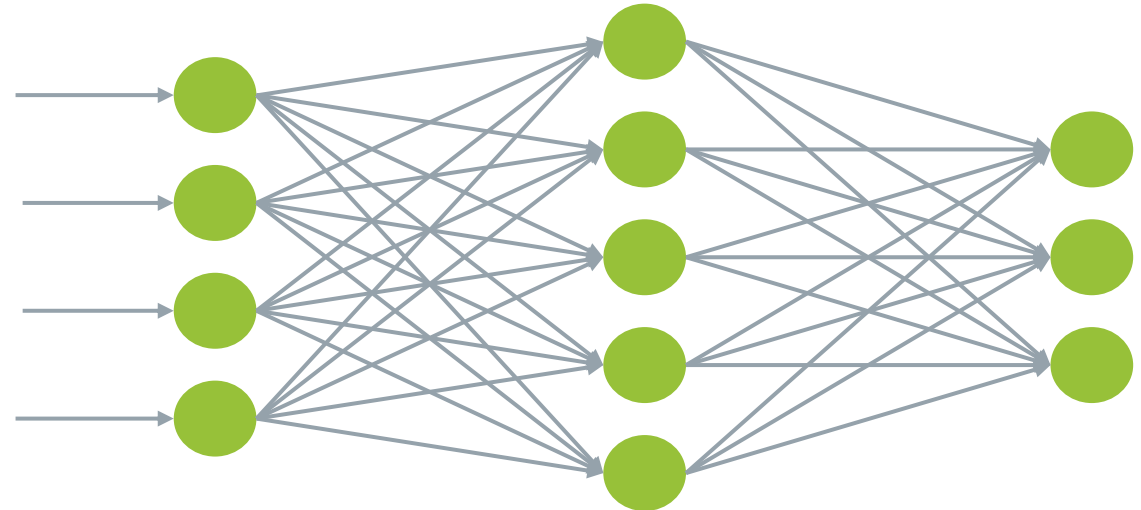
- Grundprinzipien und Ziele des maschinellen Lernens
- Grundlegende etablierte Algorithmen des maschinellen Lernens

Künstliche Neuronale Netze

- Grundlegende Definition und Parameter
- Allgemeines Vorgehen bei der Anwendung
- Faltende neuronale Netze zur Verarbeitung von Bildern
- Rekurrente Neuronale Netze zur Verarbeitung von Zeitreihen

Anwendung in der Praxis

- Datenätze, Hardware, Softwaretools, Transferlernen



Dem großen Bedarf an Daten für das Training künstlicher neuronaler Netze kann mit verschiedenen Lösungen begegnet werden.

Herausforderungen

Sehr große Anzahl an korrekt klassifizierten Daten (Labels)

Lösung

Nutzung von Datensätzen aus dem Internet

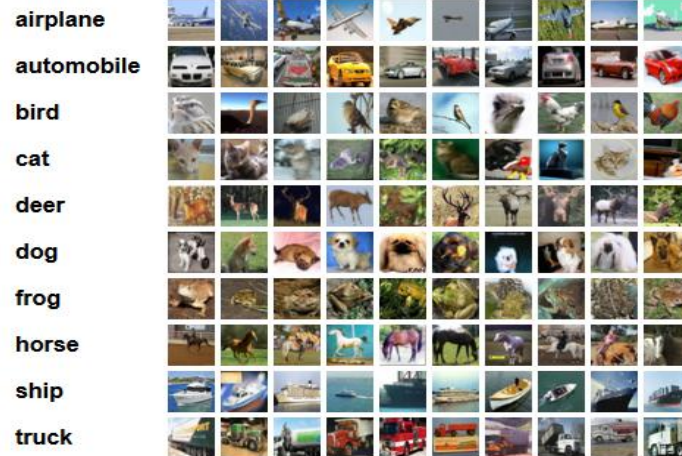
Bspw. CIFAR-Datensatz (50.000 Testbilder, 100 Klassen)
Aber Trainingsdaten $\neq$ Testdaten !!!

Data Augmentation: Bearbeitung vorhandener Trainingsdaten

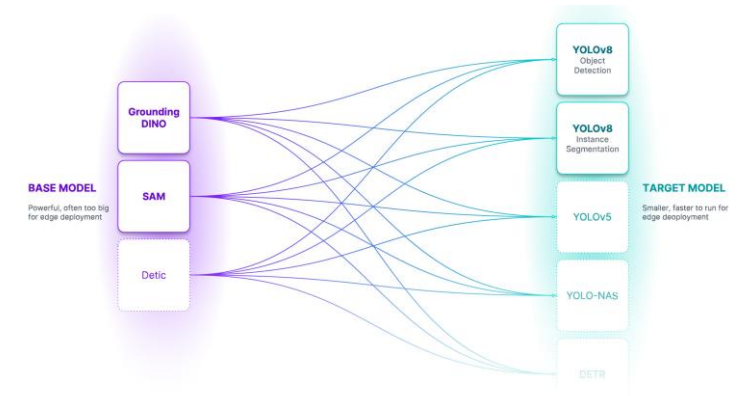
Verzerren, verrauschen, spiegeln, beschneiden ...

Pseudo-Labeling

Verwendung von großen trainierten Modellen zum automatisierten erzeugen von Labels

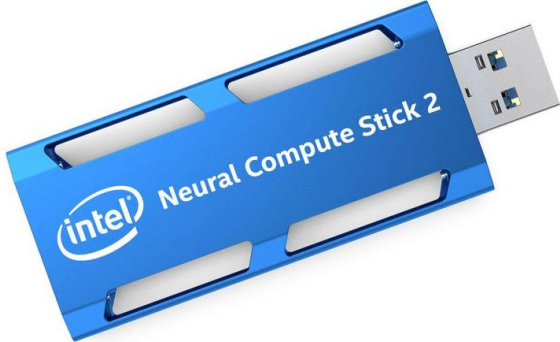


Bildquelle: <https://www.cs.toronto.edu/~kriz/cifar.html>



Bildquelle: <https://docs.autodistill.com/>

Dem großen Bedarf an Leistung beim Training und Einsatz künstlicher neuronaler Netze kann mit verschiedenen Lösungen begegnet werden.

Herausforderungen		
Große Rechenleistung oder lange Trainingsdauer , lange Klassifizierungsdauer		
Lösung		
Grafikkarten (GPU)	Effiziente Hardware Spezielle Hardware	Cloud Dienste
 Bildquelle: Nvidia	 Bildquelle: Intel	  Google Cloud
+ Hohe Leistung, Skalierbar - Großer Energiebedarf	+ Flexibler Einsatz, Skalierbar - Anpassung des Netzes nötig	+ Größte Rechenleistung, Skalierbar + Geringe Anfangskosten - Daten werden auf externen Servern gespeichert

Dem großen Bedarf an Leistung beim Training und Einsatz künstlicher neuronaler Netze kann mit verschiedenen Lösungen begegnet werden.

Herausforderungen

Große **Rechenleistung** oder lange **Trainingsdauer**, lange **Klassifizierungsdauer**

Lösung

Effiziente Software

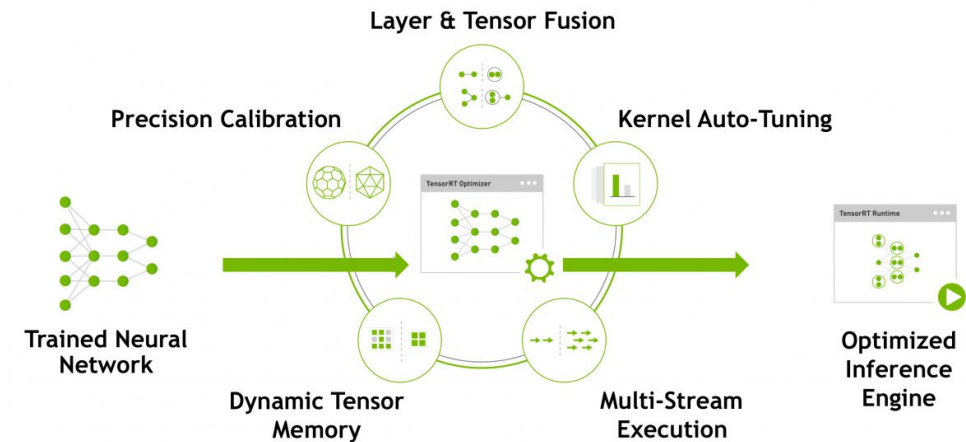
Google Tensorflow + Keras oder PyTorch (Lightning)

Optimierung des trainierten Netzes



- OpenSource SDK mit Algorithmen für KNN
- Python API
- Stand der Technik/Forschung von KNN

Siehe Übung



Bildquelle: Nvidia

Weiterführende Literatur und Links

Bücher

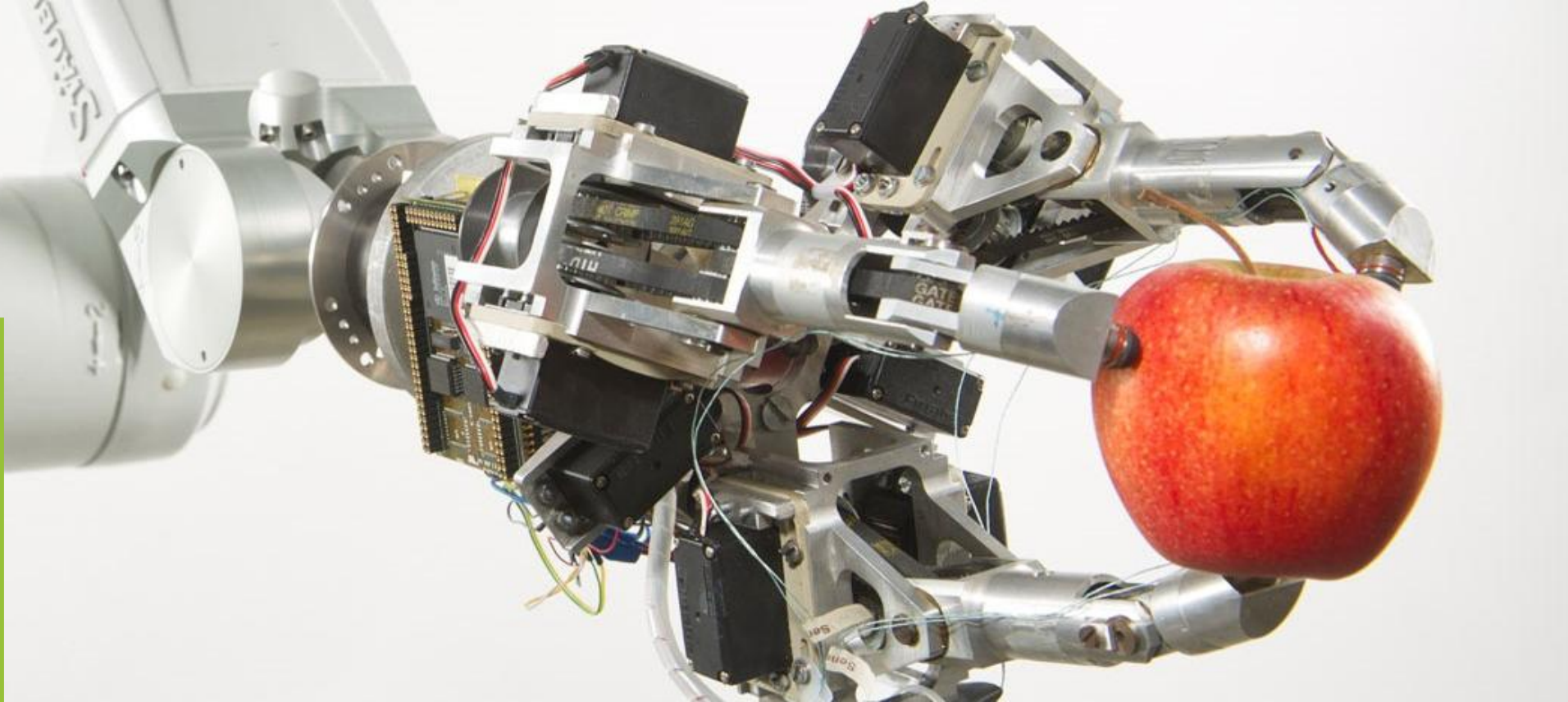
- **Deeplearning Book**, Ian Goodfellow and Yoshua Bengio and Aaron Courville, 2016 <http://www.deeplearningbook.org/>
- **Deep Learning with Python, Second Edition** François Chollet, 2021
- **Hands-On Machine Learnin with Scikit-Learn, Keras & TensorFlow, Third Edition** Aurélien Géron, 2022
- **The Hundred-Page Machine Learning Book** (<https://leanpub.com/theMLbook>)

Internet

- Stanford CS class CS231n: Convolutional Neural Networks for Visual Recognition; <http://cs231n.github.io/>
- Google Machine Learning Crash Course: <https://developers.google.com/machine-learning/crash-course/>
- edX Deep Learning Fundamentals with Keras: <https://www.edx.org/course/deep-learning-fundamentals-with-keras>

Frameworks:

- PyTorch Lightning: High Level Interface zu PyTorch <https://lightning.ai/docs/pytorch/stable/>
- PyTorch: <https://pytorch.org/>
- KERAS: The Python Deep Learning library – www.keras.io
- TensorFlow: <https://www.tensorflow.org/>



Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

DANKE